低空经济背景下的多无人机协同巡检路径优化

摘　要

针对不同等级巡检点需要重复访问、同一架无人机不能通过原地停留重复计数以及临时管制区随时间生效的多无人机协同巡检问题，本文将每次有效巡检转化为独立任务副本，建立由机队规模、系统完成时间和单机工时均衡性构成的递进式路径优化模型。

对于问题一，将任务建模为带重复巡检与最大工时约束的 Min–Max 多旅行商问题，利用总服务时间和最小生成树构造机队规模下界，再结合加权 K-means、极角扫描、合法插入、2-opt 与模拟退火搜索闭合路线。四个算例当前采用的无人机数量分别为 $4, 2, 5, 4$ 架，对应最长工作时间分别为 $8.6111, 7.5890, 8.2876, 8.6263$ h，均满足 9 小时限制。其中 Case 2 和 Case 4 的可行数量与理论下界相等，机队规模达到理论最小值；Case 1 和 Case 3 的结果为当前已验证的最小可行候选。

对于问题二，固定问题一采用的机队规模，并约束总体完成时间不劣于问题一方案，通过重载路线任务迁移、跨路线交换和 2-opt 重新分配实际工作量。四个算例的单机工时极差分别降至 $0.0174, 0.0103, 0.0238, 0.0077$ h，较问题一分别下降 $72.1\%, 9.7\%, 41.8\%, 62.6\%$，且未通过额外等待制造表面均衡。

对于问题三，以 8:00 为时间原点，在固定机队规模下建立圆形管制区的时空联合约束，构造时变可见图，同时比较安全等待与空间绕飞，并采用 ε-约束两阶段算法依次优化总体完成时间和工时极差。四个算例所得最长工作时间分别为 $8.6111, 13.1404, 9.6280, 10.9250$ h；其中 Case 2 的必要等待由基准算法的 189.297 min 降至 47.645 min。全部正式方案及机队规模敏感性结果均通过独立逐事件复核。本文所得数值为给定计算预算内的最好可行结果，不将启发式搜索结果表述为已经证明的全局最优解。

关键词： 多无人机协同巡检；Min–Max 多旅行商问题；负载均衡；时变可见图；词典序优化

目　录

一　问题重述

1.1　问题背景

低空经济的发展使无人机逐渐应用于分散设施和低空航路的日常巡检。与单次访问问题不同，本题的巡检点具有不同等级，对应不同的重复巡检次数；无人机必须从同一基地出发并返回，且同一架无人机不能通过在原地连续停留重复计数。实际执行中还会出现带有生效时段的临时圆形飞行管制区，使路线可行性同时受到空间位置和通过时刻的影响。因此，需要在有限工作时间和动态管制条件下，统筹机队规模、任务分配、访问顺序与工作负载均衡。

1.2　问题要求

问题一根据附件一给出的四组巡检点坐标和等级，在 9 小时内完成全部有效巡检并返回基地，确定各算例采用的无人机数量和协同路线，使机队规模及系统总体完成时间尽可能小。问题二固定问题一采用的机队规模，在不恶化总体完成时间的条件下重新分配任务和调整访问顺序，缩小无人机之间的工作时间极差。问题三进一步结合附件二给出的圆形管制区及生效时间窗，在固定正式机队规模下设计满足动态管制的路线，优先控制总体完成时间并改善负载均衡，同时输出详细调度方案及机队规模敏感性结果。

二　问题分析

2.1　问题一分析

问题一的核心是无人机数量与最长工作时间之间的层级关系：增加无人机通常可以缩短完成时间，但题目首先要求尽可能少的机队规模。本文将不同等级的重复巡检要求展开为独立任务副本，删除同一物理点任务副本之间的连续访问弧，并以 $\mathrm{lexmin}(N, T_{\max})$ 表示优先级。求解时先利用服务工作量和最小生成树计算数量下界，再从下界出发搜索 9 小时内可行的闭合路线；固定候选数量后，通过多起点构造、合法插入、2-opt 和模拟退火改进最长路线，并由独立程序复算任务次数和工作时间。

2.2　问题二分析

问题二在问题一调度模型的基础上进一步考虑无人机工作负载的均衡性。问题一以无人机数量和系统总体完成时间为主要目标，但仅优化最大工作时间 $T_{\max}$ 主要关注最后返回基地的无人机，不能充分反映不同无人机之间的工时差异。因此，问题二固定问题一最终采用的无人机数量，允许重新调整任务分配与访问顺序，引入最短工作时间 $T_{\min}$ 和工作时间极差 $\delta = T_{\max} - T_{\min}$，并要求新方案的总体完成时间不劣于问题一方

案。由此，前两问的整体目标可写为 $\mathrm{lexmin}(N, T_{\max}, \delta)$。求解时以问题一调度方案为初始解，通过重载路线向轻载路线的任务迁移、路线间任务交换、2-opt 和模拟退火搜索均衡方案。

2.3 问题三分析

问题三在前两问任务副本、基地闭合和有效巡检规则的基础上，引入具有确定生效时段的圆形飞行管制区，因此路径是否可行不仅取决于空间位置，还取决于无人机通过相应航段的实际时刻。本文以 8:00 为统一时间原点，将管制时刻换算为相对分钟数，通过线段–圆盘相交和时间窗重叠联合判断航段可行性。为保持三问之间的递进关系，正式方案固定采用问题一最终确定的机队规模 N_0，先控制系统总体完成时间，再在近似最优效率范围内改善工作负载均衡性；不同机队规模的结果仅用于敏感性分析。

三 模型假设

结合题目条件和实际巡检过程，作如下假设：

1. 所有无人机型号与性能相同，均以题设速度匀速飞行，忽略起飞、降落及加减速过程所需时间；
2. 将基地和巡检点视为同一平面内的点，飞行距离按欧氏距离计算，不考虑地形高度、风速等因素的影响；
3. 每次有效巡检的作业时间均为 5 min，不因巡检点等级和执行无人机的不同而改变；
4. 忽略无人机之间的碰撞、通信干扰及同一巡检点的作业容量限制，各架无人机可以独立并行执行任务；
5. 在任务执行过程中无人机不发生故障，除题目规定的时间要求外，不额外考虑电池容量、充电和维护问题；
6. 附件中巡检点及动态管制区的数据准确有效，问题三中各管制区严格按照给定时刻出现和解除，不考虑其他临时变化。

四 主要符号说明

本文使用的主要符号及其含义见表 1，其余仅在局部模型中使用的符号均在首次出现时说明。

表 1　主要符号及含义

符号	含义
N	投入使用的无人机数量
N_0	问题三正式方案采用的机队规模
r_i	巡检点 i 所需的巡检次数
U	全部巡检任务副本构成的集合
d_{ij}	巡检点 i 与巡检点 j 之间的实际距离
T_k	第 k 架无人机的工作时间
$T_{\max}$	所有无人机工作时间的最大值
$T_{\min}$	所有无人机工作时间的最小值
δ	无人机工作时间极差，$\delta = T_{\max} - T_{\min}$
$\boldsymbol{c}_r$	第 r 个动态管制圆的圆心
R_r	第 r 个动态管制圆的半径
$[\alpha_r, \beta_r)$	第 r 个动态管制区的生效时间区间
$\mathcal{F}_{ij}$	航段 (i, j) 的禁用出发时刻集合
$A_{ij}(t)$	时刻 t 从节点 i 出发至节点 j 的最早到达时刻

五　问题一的模型建立与求解

5.1　模型建立

问题一要求多架同型无人机从基地同时出发，在 9 小时内完成全部巡检任务并返回基地。由于不同等级巡检点需要被访问不同次数，且同一架无人机只有在离开某点后再次返回才能形成新的有效巡检，因此，本题可归结为一类具有重复巡检任务、单基地、同型无人机和最大工作时间约束的 Min–Max 多旅行商问题。

普通多旅行商问题主要研究如何将全部任务分配给多条由基地出发并返回基地的路径，而 Min–Max 多旅行商问题进一步以最长路径或最长工作时间为优化目标，从而尽可能提前全部任务的最终完成时刻[1]。本题中的无人机数量并非预先给定，因此采用先最小化车辆数量、再优化路径代价的两阶段思想[2-3]，将总体目标表示为

$$\operatorname{lexmin}\left(N, T_{\max}\right). \tag{五.1}$$

其中，N 为投入使用的无人机数量，$T_{\max}$ 为全部无人机工作时间的最大值。无人机数量具有绝对优先级，只有在 N 取到最小可行值后，才继续优化 $T_{\max}$。因此，本文

不采用 $\min \alpha N + \beta T_{\max}$ 的加权目标，以避免因权重设置不当而通过增加无人机数量缩短完成时间。

任务副本化与有效巡检规则。设全部物理巡检点构成集合 $V = \{1, 2, \ldots, n\}$，基地记为节点 0，其坐标为 $(x_0, y_0) = (0, 0)$。根据巡检等级，点 i 所需的巡检次数为

$$r_i = \begin{cases} 3, & i \text{ 为 I 级巡检点}, \\ 2, & i \text{ 为 II 级巡检点}, \\ 1, & i \text{ 为 III 级巡检点}, \end{cases} \qquad M = \sum_{i \in V} r_i.$$

其中，M 表示全部巡检任务数。为区分同一物理点上的多次巡检，将每次巡检要求转化为一个独立任务副本。对于物理巡检点 i，建立副本集合 $U_i = \{(i,1), (i,2), \ldots, (i,r_i)\}$；全部任务副本构成集合 $U = \bigcup_{i \in V} U_i$，且有 $|U| = M$。

定义映射 $p(u)$ 表示任务副本 u 对应的物理巡检点。例如，若点 7 为 I 级巡检点，则将其展开为 $(7,1), (7,2), (7,3)$ 三个任务副本。不同副本可以分配给同一架或不同无人机，模型不设置同一时刻只能有一架无人机访问某点的互斥约束。

同一架无人机不能通过原地停留重复完成同一物理点的多个任务副本。令扩展节点集合为 $\bar{U} = U \cup \{0\}$，定义可行有向弧集合为

$$\mathcal{A} = \left\{ (u, v) \in \bar{U} \times \bar{U} : u \neq v,\ \neg\left(u, v \in U \text{ 且} p(u) = p(v)\right) \right\}. \tag{五.2}$$

式 (五.2) 删除了同一物理点不同任务副本之间的直接连接。例如，路线 $0 \to 7^{(1)} \to 7^{(2)} \to 0$ 不合法，而路线 $0 \to 7^{(1)} \to 12^{(1)} \to 7^{(2)} \to 0$ 合法。因此，该规则只禁止同一架无人机在原地连续重复计数，不禁止其离开后再次返回，也不限制不同无人机分别完成同一点的巡检任务。

距离和工作时间。附件中的坐标每 1 个单位对应实际距离 100 m，即 0.1 km，因此物理巡检点 i 与 j 之间的实际欧氏距离为 $d_{ij} = 0.1\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ km。令 $p(0) = 0$，则对于任意可行弧 $(u, v) \in \mathcal{A}$，其飞行距离为 $d_{uv} = d_{p(u),p(v)}$，按照无人机巡航速度 55 km/h 计算，相应飞行时间为 $\tau_{uv} = d_{uv}/55$。每完成一个任务副本需要 5 分钟巡检作业，因此单次服务时间为 $t_s = 5/60 = 1/12$ h。I、II、III 级巡检点分别展开为 3、2、1 个任务副本。附件坐标和管制圆半径采用同一坐标尺度；程序计算时始终保留完整浮点精度，仅在结果表中将工作时间保留 4 位小数，返回时刻则由未舍入的秒级结果换算得到。

设无人机 k 的路线为 $R_k = (0, u_1, u_2, \ldots, u_{m_k}, 0)$，其中 m_k 为该无人机完成的任务副本数量，则其工作时间为

$$T_k = \frac{1}{55}\left(d_{0u_1} + \sum_{q=1}^{m_k - 1} d_{u_q u_{q+1}} + d_{u_{m_k} 0} \right) + \frac{m_k}{12}. \tag{五.3}$$

式 (五.3) 的第一项为飞行时间，第二项为全部任务副本的服务时间。即使多个任务副本对应同一物理点，每完成一次有效巡检仍需单独计入 5 分钟服务时间。

由于所有无人机同时起飞，最后一架无人机返回基地的时刻即为全部任务的完成时刻。因此，

$$T_{\max} = \max_{k=1,\dots,N} T_k, \qquad T_k \le 9 \quad (\forall k). \tag{五.4}$$

无人机数量理论下界。全部 M 个任务副本必然产生 $M/12$ 小时的服务时间。由于每架无人机最多工作 9 小时，仅考虑服务时间即可得到 $N \ge \lceil M/108 \rceil$。

进一步将基地与所有不同物理巡检点构成完全无向图，并以实际欧氏距离 d_{ij} 为边权。设该图的最小生成树长度为 L_{MST}。任意能够完成全部任务的无人机路线集合，其飞行边的并集必须将基地与所有巡检点连接起来，故总飞行距离不小于 L_{MST}。由此得到全部无人机总工作量下界 $W_{\mathrm{LB}} = M/12 + L_{\mathrm{MST}}/55$，以及无人机数量理论下界

$$N_{\mathrm{LB}} = \left\lceil \frac{\dfrac{M}{12} + \dfrac{L_{\mathrm{MST}}}{55}}{9} \right\rceil. \tag{五.5}$$

此外，至少有一架无人机需要到达距离基地最远的巡检点并返回。结合最远点下界和平均工作量下界，给定无人机数量 N 时有

$$T_{\max}^{\mathrm{LB}}(N) = \max\left\{ \frac{W_{\mathrm{LB}}}{N}, \max_{i \in V} \left(\frac{2d_{0i}}{55} + \frac{1}{12} \right) \right\}. \tag{五.6}$$

若 $T_{\max}^{\mathrm{LB}}(N) > 9$，则 N 架无人机一定无法完成任务；若该下界不超过 9 小时，则只能说明当前数量尚未被理论下界排除，不能据此直接判定其一定可行。

固定无人机数量的数学规划模型。给定候选无人机数量 N，令无人机集合为 $K = \{1, 2, \dots, N\}$。定义二元变量 y_u^k 表示任务副本 u 是否由无人机 k 完成，二元变量 x_{uv}^k 表示无人机 k 是否沿可行弧 (u, v) 飞行，即

$$y_u^k = \begin{cases} 1, & \text{任务副本}u \text{ 由无人机}k \text{ 完成}, \\ 0, & \text{否则}, \end{cases} \qquad x_{uv}^k = \begin{cases} 1, & \text{无人机}k \text{ 沿弧}(u,v) \text{ 飞行}, \\ 0, & \text{否则}. \end{cases}$$

每个任务副本必须且只能由一架无人机完成；若任务副本被分配给某架无人机，则该无人机必须恰好进入并离开该任务一次；同时，每架投入使用的无人机只能从基地出发和返回一次。上述条件统一写为

$$\begin{aligned} &\sum_{k \in K} y_u^k = 1, && u \in U, \\ &\sum_{v:(v,u) \in \mathcal{A}} x_{vu}^k = y_u^k, && u \in U,\ k \in K, \\ &\sum_{v:(u,v) \in \mathcal{A}} x_{uv}^k = y_u^k, && u \in U,\ k \in K, \\ &\sum_{v:(0,v) \in \mathcal{A}} x_{0v}^k = 1, \qquad \sum_{u:(u,0) \in \mathcal{A}} x_{u0}^k = 1, && k \in K. \end{aligned} \tag{五.7}$$

仅设置出入度约束仍可能产生不经过基地的独立子回路。为此，引入非负单商品流变量 f_{uv}^k。每个被无人机 k 完成的任务副本消耗一个单位任务流，而基地发出与该无人机承担任务数量相等的总流量：

$$\begin{aligned} &0 \le f_{uv}^k \le M x_{uv}^k, && (u,v)\in\mathcal{A},\ k\in K,\\ &\sum_{v:(v,u)\in\mathcal{A}} f_{vu}^k - \sum_{v:(u,v)\in\mathcal{A}} f_{uv}^k = y_u^k, && u\in U,\ k\in K,\\ &\sum_{v:(0,v)\in\mathcal{A}} f_{0v}^k - \sum_{u:(u,0)\in\mathcal{A}} f_{u0}^k = \sum_{u\in U} y_u^k, && k\in K. \end{aligned} \tag{五.8}$$

该约束组可以排除与基地不连通的任务子回路，保证每架无人机形成一条完整的基地闭合路线。

在数学规划模型中，无人机 k 的工作时间、总体完成时间和 9 小时限制统一表示为

$$\begin{aligned} T_k &= \sum_{(u,v)\in\mathcal{A}} \tau_{uv} x_{uv}^k + \frac{1}{12}\sum_{u\in U} y_u^k, && k\in K,\\ T_k &\le C_{\max}, && k\in K,\\ C_{\max} &\le 9. \end{aligned} \tag{五.9}$$

其中，$C_{\max}$ 表示系统总体完成时间。固定无人机数量 N 后，模型以 $\min C_{\max}$ 为目标。记固定 N 时得到的最优目标值为 $C^*(N) = \min \max_{k\in K} T_k$，则真正的最小无人机数量定义为

$$N_{\min} = \min\left\{N\in\mathbb{Z}_+ : C^*(N)\le 9\right\}, \qquad N_{\mathrm{LB}} \le N_{\min}. \tag{五.10}$$

若在 $N = N_{\mathrm{LB}}$ 时找到满足 9 小时约束的可行方案，则上下界相等，可以严格确定最小无人机数量；若找到的可行数量大于理论下界，则还需证明更小数量不可行，才能严格认定该数量为 $N_{\min}$。

5.2 求解方法

(1) 求解流程与初始解。问题一按照式 (五.1) 的优先关系分两阶段求解：先从数量下界 N_{LB} 出发搜索满足 9 小时限制的无人机数量，再固定 N 优化最大工作时间 $T_{\max}$。对每个给定的 N，程序采用多个随机起点，交替使用加权 K-means 和极角扫描划分任务区域，并以最近邻法和 2-opt 构造各区域的初始闭合路线。对于重复巡检任务，按式 (五.11) 计算插入增量，并排除与同一物理点相邻的位置：

$$\Delta d = d_{ai} + d_{ib} - d_{ab}. \tag{五.11}$$

由此得到满足巡检次数要求且不存在同点连续访问的初始方案。数量搜索的总体过程如图 1 所示。

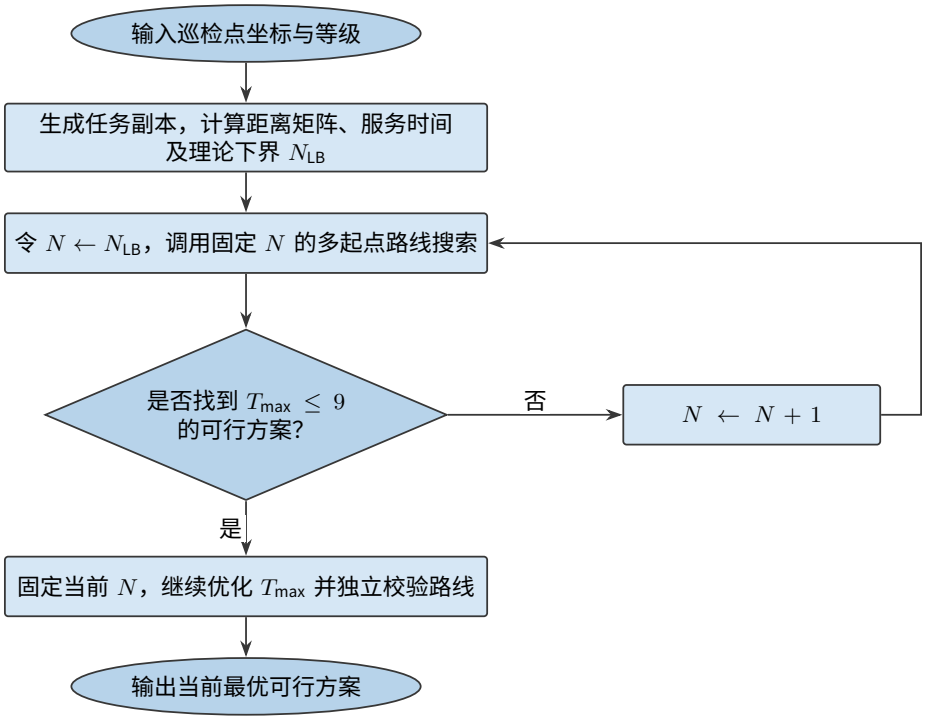


图 1　问题一无人机数量与路径的两阶段搜索流程

（2）路线优化与结果校验。在初始方案上，程序通过任务迁移、任务交换和 2-opt 构造邻域，并采用模拟退火搜索降低 $T_{\max}$。对使目标值增加 $\Delta T > 0$ 的候选方案，以概率

$$P = \exp\left(-\frac{\Delta T}{\tau}\right) \tag{五.12}$$

予以接受，其中 τ 为温度参数。退火结束后，再针对最长路线执行确定性平衡修复，并保留全部随机起点中 $T_{\max}$ 最小的方案。若当前 N 找到满足 $T_{\max} \leq 9$ 的路线，则记录结果并继续优化；否则令 $N \leftarrow N+1$。最后独立复算各路线，检查巡检次数、同点连续访问、基地闭合和 9 小时限制，并将详细调度方案写入 `result1.xlsx`。固定无人机数量下的完整优化流程及相关源程序见附录 B。

各算法操作的作用归纳于表 2。

表 2　问题一各算法操作及其主要作用

算法操作	在本题中的主要作用
加权 K-means	根据空间位置和巡检次数形成地理上较集中的初始任务区域
极角扫描	从不同方向切分任务点，增加初始路线结构的多样性
最近邻与 2-opt	快速构造闭合路线并减少明显绕行和路线交叉
合法贪心插入	补充重复巡检任务，同时禁止同一物理点连续出现
任务迁移与交换	从最长路线向其他路线转移工作量，降低最大工作时间
模拟退火与多起点	接受少量劣化移动并重复搜索，降低陷入局部最优的可能性
确定性平衡修复	对当前最好方案进行最后的任务转移、交换和路线缩短

5.3　结果与解释

按照赛题规定的展示格式，四个测试算例的无人机数量及单架无人机最长、最短工作时间见表 3。

表 3　问题一无人机数量与完成时间计算结果

测试算例	无人机数量 N	单架无人机最长工作时间 $T_{\max}$（h）	单架无人机最短工作时间 $T_{\min}$（h）
Case 1	4	8.6111	8.5488
Case 2	2	7.5890	7.5775
Case 3	5	8.2876	8.2468
Case 4	4	8.6263	8.6058

四个算例均满足 $T_{\max} < 9$ h，说明表中方案能够在规定时间内完成全部巡检任务。理论分析得到 Case 1–Case 4 的无人机数量下界依次为 3、2、4、4；其中 Case 2 和 Case 4 的计算数量与下界相等，可以确定 $N_{\min} = 2$ 和 $N_{\min} = 4$。对于 Case 1 和 Case 3，当前结果分别给出 4 架和 5 架无人机的调度方案，但启发式算法在给定计算时间内未找到 3 架和 4 架方案并不等价于严格不可行，因此只能得到 $3 \le N_{\min} \le 4$ 和 $4 \le N_{\min} \le 5$。此外，表中 $T_{\max}$ 是多起点搜索得到的当前最好结果，未使用精确求解器证明其为固定 N 下的全局最优值。

六　问题二的模型建立与求解

6.1　递进模型与均衡目标

问题二完整继承问题一的任务副本集合、工作时间计算方法和路径可行性约束，包括每个任务副本恰好完成一次、无人机从基地出发并最终返回基地、同一物理巡检点禁止连续巡检、允许离开后再次返回以及单架无人机工作时间不超过 9 小时等约束。与问题一不同，问题二不再搜索无人机数量，而是固定

$$N = N^{(1)}, \qquad N^{(1)} = (4, 2, 5, 4), \tag{六.1}$$

其中，向量各分量依次对应 Case 1–Case 4。鉴于 Case 1 和 Case 3 的当前数量尚未被严格证明为全局最少数量，本文将 $N^{(1)}$ 表述为“问题一最终采用的无人机数量”，而不统一记作已经严格证明的 $N_{\min}$。

问题二不允许通过人为等待延长较短路线。均衡性改善只来源于任务副本重新分配和访问顺序调整，各无人机工作时间仍仅由飞行时间和有效巡检服务时间组成。主方案同时固定 $N = N^{(1)}$、取 $\varepsilon = 0$，即不增加无人机、不放宽问题一当前完成时间，也不人为增加等待。

为刻画实际工作时间的均衡程度，定义

$$T_{\max} = \max_{k \in K} T_k, \qquad T_{\min} = \min_{k \in K} T_k, \qquad \delta = T_{\max} - T_{\min}. \tag{六.2}$$

由于恒有 $T_{\max} \geq T_{\min}$，题目中的绝对值可直接写成式 (六.2)。车辆路径均衡问题中常以最长路线时间或最长、最短路线时间之差衡量工作负载差异[4-5]。本文采用“实际工作时间极差”，而不是任务数量差或飞行距离差。引入辅助变量 U, L，可将其线性表示为

$$\begin{aligned} U &\geq T_k, \qquad k \in K, \\ L &\leq T_k, \qquad k \in K, \\ \delta &= U - L. \end{aligned} \tag{六.3}$$

在最小化 δ 时，最优解处有 $U = T_{\max}$、$L = T_{\min}$。但若只最小化极差，可能通过牺牲总体完成时间获得表面上的均衡。为保持问题一的优先目标，设

$$C_{\text{ref}} = T^{(1)}_{\max}, \qquad T^{(2)}_{\max} \leq C_{\text{ref}} + \varepsilon. \tag{六.4}$$

主方案取 $\varepsilon = 0$，即第二问的总体完成时间不得超过问题一当前最好可行方案。这相当于采用 ε-约束思想，在保持主要目标既有水平的条件下优化次级目标[6]。四个算例的 C_{ref} 依次为

$$(8.6111,\ 7.5890,\ 8.2876,\ 8.6263)\ \text{h}.$$

因此，前两问可以统一表示为

$$\operatorname{lexmin}\left(N, T_{\max}, \delta, \sum_{k\in K} T_k\right). \tag{六.5}$$

在第二问中，前两层已由式 (六.1) 和式 (六.4) 固定，故对满足约束的候选方案采用

$$F(S) = \left(\delta(S), T_{\max}(S), \sum_{k\in K} T_k(S)\right) \tag{六.6}$$

进行词典序比较。总工作时间只作为前三项相同情况下的平局判别，以减少无效绕行。

6.2 均衡优化算法

程序直接读取问题一通过独立校验的调度方案 $S^{(1)}$，固定 $N = N^{(1)}$ 并令 $C_{\mathrm{ref}} = T_{\max}^{(1)}$。每次迭代先按工作时间识别重载路线与轻载路线，再从重载路线向轻载路线迁移一个任务，或交换两条路线中的任务。发生改变的路线随后执行合法 2-opt，以消除明显绕行。候选方案必须同时满足巡检次数、基地闭合、同点不连续、9 小时限制以及 $T_{\max} \le C_{\mathrm{ref}}$，否则直接拒绝。

若候选方案的式 (六.6) 词典序减小，则予以接受；对少量暂时劣化的方案按照模拟退火概率接受，以降低陷入局部最优的可能性。程序使用多个随机种子重复搜索，最后系统枚举重载路线与轻载路线间的单任务迁移，并对各路线充分执行 2-opt。上述过程不再改变无人机数量，也不重新构造问题一的任务副本模型，重点只在既有完成时间上界内重新分配真实巡检工作量。

需要特别说明的是，本文不允许无人机通过在基地、巡检点或航路中额外等待来增加工作时间。所有方案的等待时间均为 0，T_k 只由实际飞行时间和有效巡检服务时间构成；路线调整后继续执行距离缩短操作。因此，均衡改善来自真实任务的重新分配和访问顺序优化，而不是人为延长短路线。该处理也避免了极差型非单调均衡指标可能诱发非必要绕行的问题[4]。

6.3 计算结果与分析

问题二的无人机数量、最大工作时间、最小工作时间和工作时间极差见表 4。全部算例均使用问题一最终采用的无人机数量，且满足 $T_{\max}^{(2)} \le C_{\mathrm{ref}} < 9\mathrm{h}$。

表 4　问题二结果展示表

测试算例	无人机数量 N	单架无人机最长工作时间 $T_{\max}$ (h)	单架无人机最短工作时间 $T_{\min}$ (h)	$\delta = \lvert T_{\max} - T_{\min} \rvert$ (h)
Case 1	4	8.6111	8.5937	0.0174
Case 2	2	7.5841	7.5737	0.0103
Case 3	5	8.2824	8.2586	0.0238
Case 4	4	8.6197	8.6120	0.0077

为量化均衡改善效果，记问题一、问题二的工作时间极差分别为 δ_1, δ_2，定义极差下降量和下降比例为

$$\Delta\delta = \delta_1 - \delta_2, \qquad \eta_\delta = \frac{\delta_1 - \delta_2}{\delta_1} \times 100\%. \tag{六.7}$$

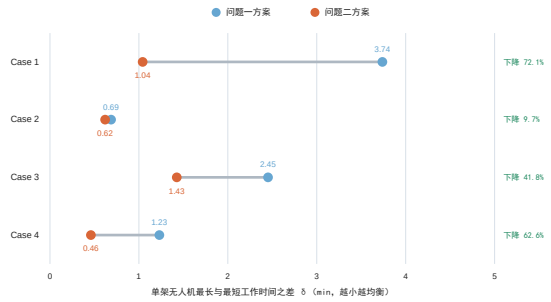
计算结果见表 5。

表 5　问题一与问题二工作时间极差对比

测试算例	δ_1/min	δ_2/min	下降比例
Case 1	3.74	1.04	72.1%
Case 2	0.69	0.62	9.7%
Case 3	2.45	1.43	41.8%
Case 4	1.23	0.46	62.6%

注：极差及下降比例均由程序输出的未舍入数据计算，表中分钟数仅作显示舍入。

图 2　问题一与问题二工作时间极差对比



由表 5 和图 2 可见，问题二所得方案在保持无人机数量不变的基础上，进一步缩小了不同无人机之间的工作时间极差。按照程序输出的未舍入数据计算，Case 1、Case 3 和 Case 4 的极差分别下降 72.1%、41.8% 和 62.6%；Case 2 在问题一中已经较为均衡，极差仍由约 0.69 分钟下降至 0.62 分钟，下降 9.7%。优化后四个算例的工作时间极差均不超过 1.43 分钟，说明所构造方案具有较好的负载均衡性。与此同时，Case 2–Case 4 的 $T_{\max}$ 还略有下降，Case 1 保持原完成时间上界，说明均衡改善没有以牺牲总体完成时间为代价。

正文不逐行展开较长的巡检点序列。四个算例中每架无人机的基地闭合路线、飞行距离、巡检次数和工作时间均保存在支撑材料 `result2.xlsx`，可复算的完整结构化路线保存在 `q2_solution.json`；相应程序入口为 `q2_solver.py`。

七　问题三的模型建立与求解

7.1　模型建立

问题三并非管制信息随机到达的在线调度，而是在管制区域及其生效时段均预先已知的条件下，对前两问模型增加绝对时刻和时变通行约束，属于确定性时间依赖路径问题[7]。题目未明确说明第三问是否允许重新配置无人机数量；若将机队规模视为不受约束的决策变量，而目标仅为缩短完成时间，则会产生不断增加无人机的退化倾向。为保持三问的递进关系，本文将问题一最终采用的机队规模视为系统已有配置，正式方案固定为 $N_0 = (4, 2, 5, 4)$。这是本文用于消除题意歧义的建模约定，并非题目明文规定；增加无人机的结果仅用于第 8 节敏感性分析。

任务副本、巡检次数、基地往返以及同一物理点不得连续巡检等约束保持不变。以 8:00 为 $t = 0$，若附件钟表时刻为 $h{:}m$，则相对时刻按 $t_{\mathrm{rel}} = 60(h - 8) + m$ 换算，例如 8:00 $\mapsto$ 0、10:30 $\mapsto$ 150、17:00 $\mapsto$ 540。问题一中的 9 小时限制只用于确定机队规模，问题三原文未再次规定任务必须在 9 小时内完成，因此本文不将 17:00 作为任务截止时刻。17:00 仅表示相关动态管制区在该时刻解除，无人机可以在此后继续执行巡检任务并返回基地。

将第 r 个动态圆形管制区记为 $Z_r = (\boldsymbol{c}_r, R_r, [\alpha_r, \beta_r))$。其中 $\boldsymbol{c}_r$、R_r 分别为圆心和半径。时间窗采用左闭右开的半开区间：开始时刻 α_r 计入管制，结束时刻 β_r 不再计入，从而避免端点重复判定。Case 4 中 Z_8 的开始和结束时刻均为 17:00，故其时间窗为 $[540, 540) = \varnothing$，不产生实际管制约束；本文不将其解释为“从 17:00 持续至任务结束”，也不人为补充管制时长。对航段 (i, j)，设正常飞行时间为 τ_{ij}，程序解析求得该线段进入、离开圆 r 的参数 λ^{-}_{ijr} 与 λ^{+}_{ijr}。模型中的核心时空约束及通行函数集中写为

$$
\begin{aligned}
&\boldsymbol{p}_{ij}(\lambda) = \boldsymbol{p}_i + \lambda(\boldsymbol{p}_j - \boldsymbol{p}_i), \quad 0 \le \lambda \le 1, \\
&t + \lambda\tau_{ij} \in [\alpha_r, \beta_r) \implies \|\boldsymbol{p}_{ij}(\lambda) - \boldsymbol{c}_r\|_2 > R_r, \\
&\mathcal{F}_{ijr} = [\alpha_r - \lambda^{+}_{ijr}\tau_{ij},\ \beta_r - \lambda^{-}_{ijr}\tau_{ij}), \quad \mathcal{F}_{ij} = \bigcup_r \mathcal{F}_{ijr}, \\
&D_{ij}(t) = \min\{s \ge t : s \notin \mathcal{F}_{ij}\}, \quad A_{ij}(t) = D_{ij}(t) + \tau_{ij}.
\end{aligned}
\tag{七.1}
$$

式 (七.1) 表明，一条直线航段在几何上穿过管制圆并不意味着必然违规；只有无人机通过圆内部分的实际时间与管制窗重叠时，才形成时空冲突。当同一航段受多个管制圆影响时，程序将各圆产生的非空禁用区间按开始时刻排序，合并相交或相邻区间后组成 $\mathcal{F}_{ij}$，再从计划时刻起寻找第一个不属于该集合的出发时刻 $D_{ij}(t)$。

服务过程同样受动态管制约束。设任务 i 的服务开始时刻为 s_i、服务时间为 $t_s = 5$ min；若 $\|\boldsymbol{p}_i - \boldsymbol{c}_r\|_2 \le R_r$，必须满足 $[s_i, s_i + t_s) \cap [\alpha_r, \beta_r) = \varnothing$，即完整 5 分钟服务区间均须安全，不能只检查到达或服务开始时刻。若原计划航段当前不可通行，则推迟至最早可行出发时刻；若当前节点在计划等待区间内也不安全，则沿既定访问序列向前回溯，

将等待前移至最近一个能在整个区间保持安全的已访问节点，并重新递推后续时刻。问题三只允许这种满足管制要求所必需的等待，不允许为了减小 δ 任意延长短路线；等待时间计入工作时间，在基地等待也从 8:00 开始计时。图 3 给出了这一联合判定过程。

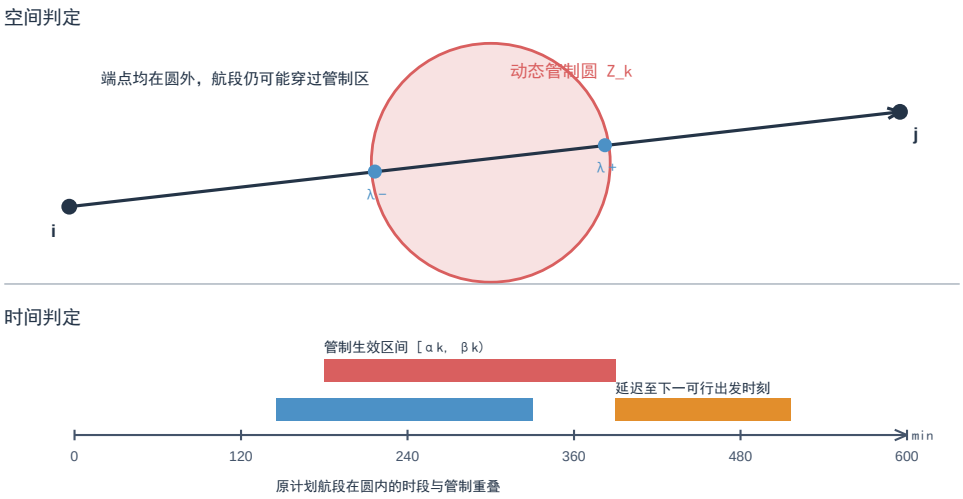


图 3　航段与动态圆形禁飞区的时空耦合判定

设无人机 k 的节点序列为 $(v_{k,0},\dots,v_{k,m_k})$，从 8:00 开始递推到达、服务和离开时刻。其工作时间由实际飞行、有效巡检和必要安全等待组成。由于 T_k^{fly}、T_k^{service} 以小时计而 T_k^{wait} 以分钟计，故 $T_k = T_k^{\mathrm{fly}} + T_k^{\mathrm{service}} + T_k^{\mathrm{wait}}/60$。并记 $T_{\max} = \max_k T_k$、$T_{\min} = \min_k T_k$、$\delta = T_{\max} - T_{\min}$。对需要重复巡检的点，程序为每次巡检建立唯一任务副本，从而保证移动、交换后仍能逐一核对任务次数。

7.2　求解方法

直飞–等待基准算法 A。

算法 A 首先读取问题二已通过校验的路线，解析计算直线航段与各管制圆的相交参数及禁用出发区间。给定任务序列后，内层评价器按照访问顺序递推最早可行时刻：航段若在管制生效时穿圆，则在最近的安全节点等待至直飞可行；若目标点的 5 分钟服务区间与管制窗冲突，则同样将必要等待前移。算法 A 不增设圆边界绕飞节点，其空间避让依靠调整访问顺序和跨无人机重分配实现，因此属于“直飞–必要等待”的快速基准方法。

外层在任务迁移、跨路线交换和路线片段反转等邻域之间切换，并用多起点模拟退火跳出局部最优，候选解按 $(T_{\max}, \delta, \sum_k T_k)$ 的字典序更新。数值计算中每个算例使用 5 个随机起点，每个起点最多迭代 25000 次；温度由 0.12 逐步降至 0.002。完成全部随机起点或达到每个算例 35 秒总时间预算时停止，并输出当前搜索范围内的最好可行方案。算法 A 的作用是提供稳定、可复算的比较基准，而不是填写第三问的最终表格。

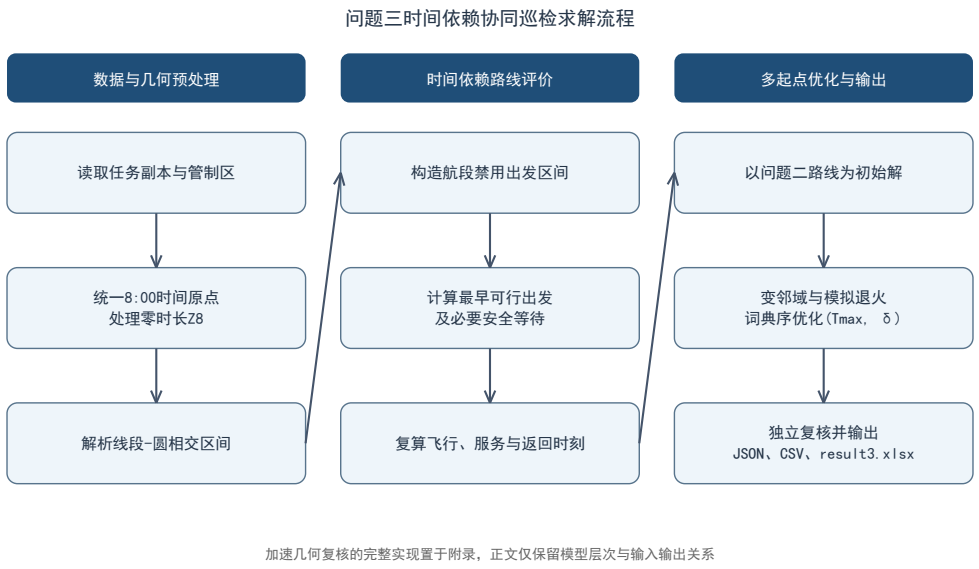


图 4　基准算法 A 的时间依赖路线求解流程

时变可见图两阶段算法 B。

算法 B 保留算法 A 的解析冲突检查和最早时刻递推，并在航段评价中增加空间绕飞选择。记 n_b 为每个有效管制圆的外偏候选节点数，本次取 $n_b = 16$；ρ 为附加安全裕度，本次取 $\rho = 0$；$\widehat{R}_r$ 为外偏采样半径。为避免候选节点落在原禁飞边界上，并防止相邻采样点之间的弦切入圆内，程序采用 $\widehat{R}_r = (R_r + \rho)\sec(\pi/n_b) + 10^{-6}$。即使 $\rho = 0$，几何外偏量 $R_r[\sec(\pi/16) - 1] + 10^{-6}$ 仍为正。

本文将管制圆视为闭圆盘，圆周边界同样属于管制区域；几何判断对判别式和点到线段距离采用统一数值容差，相切按接触管制圆处理。候选节点仅用于构造时变可见图，任意两节点间的可见边仍须在原半径 R_r 下通过解析线段–圆盘检查，不满足安全条件的边均予删除。算法同时比较“安全等待后直飞”和“沿外偏节点改道”两类通行方式，并对最终折线路径的每一小段重新进行时空联合验证。由于时变可见图只使用有限个外偏节点，所得路线是连续圆障碍最短路的离散近似，本文不声称其为连续空间中的全局最短路径。

外层搜索除迁移、交换和片段反转外，还加入跨路线片段交换，并将问题二路线、算法 A 路线及随机扰动路线共同作为初始解。为体现总体完成时间优先，并允许在近似最优效率范围内进一步改善负载均衡性，算法 B 采用基于 ε-约束的近似字典序两阶段准则

$$
\begin{aligned}
&T_{\max}^{B,*} = \min_{S \in \Omega_3(N_0)} T_{\max}(S), \\
&\min_{S \in \Omega_3(N_0)} \left(\delta(S), \sum_k T_k(S) \right) \quad \text{s.t.} \quad T_{\max}(S) \le (1 + \varepsilon_T) T_{\max}^{B,*}, \qquad \varepsilon_T = 0.005. \qquad (七.2)
\end{aligned}
$$

其中 $\Omega_3(N_0)$ 表示固定机队规模且满足全部巡检、基地闭合和动态禁飞约束的可行

方案集合。本文取 $\varepsilon_T = 0.005$，表示均衡优化只允许总体完成时间相对第一阶段当前最好值增加不超过 0.5%，从而给效率损失设置直观且较严格的上界；该参数是效率与均衡之间的建模取舍，不是数值误差。第二阶段若发现更小的 $T_{\max}$，则立即回代更新 $T_{\max}^{B,*}$ 并重新执行均衡搜索，使效率阈值始终以当前最好完成时间为基准。每个阶段最多迭代 1200 次，基础搜索每个“算例–机队规模”采用 30 秒预算；回代最多追加 3 轮，每轮最多 10 秒。达到迭代或时间上限，或回代后未再发现更小的 $T_{\max}$ 时停止。最终结果均称为“当前搜索范围内最好可行方案”，不声称已经证明全局最优。

表 6　算法 A 与算法 B 的求解机制比较

比较内容	算法 A	算法 B
航段方式	直飞或安全等待	直飞、安全等待或空间绕飞
几何结构	原任务节点	原任务节点与圆外偏置节点
动态评价	禁用出发区间与时刻递推	时变可见图与逐段时刻递推
初始解	问题二路线	问题二路线、算法 A 路线及随机扰动
邻域操作	迁移、交换、反转	增加跨路线片段交换
优化准则	字典序基准搜索	ε-约束两阶段优化
论文作用	对照基准	固定 N_0 的正式方案

7.3　计算结果与分析

按照上述约定，第三问正式结果采用算法 B 在固定机队规模 $N_0 = N^{(1)} = (4, 2, 5, 4)$ 下得到的均衡方案，列于表 7；该表用于填写赛题所给表 4。算法 A 仅作为基准方法，不同机队规模结果仅作为敏感性分析。表中工作时间从 8:00 起算，允许在 17:00 后继续执行任务并返回基地。

表 7　问题三结果展示表（算法 B 固定 N_0 的正式方案）

测试算例	无人机数量 N	单架无人机最长工作时间 $T_{\max}$ (h)	单架无人机最短工作时间 $T_{\min}$ (h)	$\delta = \lvert T_{\max} - T_{\min} \rvert$ (h)
Case 1	4	8.6111	8.5937	0.0174
Case 2	2	13.1404	13.1401	0.0003
Case 3	5	9.6280	9.5822	0.0458
Case 4	4	10.9250	10.8597	0.0652

表中工作时间以小时表示并保留 4 位小数，最晚返回时刻则根据程序保存的未舍入秒级结果计算，因此直接使用表中四舍五入后的小时数换算时可能出现约 1 秒差异。

表 8　固定 N_0 时算法 A 与算法 B 的结果对比

算例	N_0	$T_{\max}^{A}$/h	$T_{\max}^{B,*}$/h	$T_{\max}^{B}$/h	δ^{A}/h	δ^{B}/h
Case 1	4	8.6111	8.6111	8.6111	0.0174	0.0174
Case 2	2	13.3779	13.1404	13.1404	0.0146	0.0003
Case 3	5	9.6086	9.6086	9.6280	0.1237	0.0458
Case 4	4	10.8711	10.8711	10.9250	0.6808	0.0652

表 8 中 $T_{\max}^{B,*}$ 为算法 B 第一阶段得到的效率基准，$T_{\max}^{B}$ 为第二阶段均衡后的正式值。Case 1 未发生实际管制冲突，两种算法结果一致。Case 2 中，空间绕飞与任务重排使完成时间由 13.3779 小时降至 13.1404 小时，必要等待由 189.297 分钟降至 47.645 分钟，同时极差降至约 0.02 分钟。Case 3、Case 4 的第二阶段分别在第一阶段最好完成时间上增加约 0.20% 和 0.50%，均未超过 0.5% 阈值，而极差分别缩小约 63.0% 和 90.4%。这说明在严格限制效率损失后，算法 B 能显著改善固定机队下的工作负载均衡。

表 9　固定 N_0 时必要等待与绕行结果对比

算例	算法 A 必要等待/min	算法 B 必要等待/min	算法 B 绕行航段数
Case 1	0.000	0.000	0
Case 2	189.297	47.645	12
Case 3	59.023	19.671	1
Case 4	0.000	0.000	1

表 9 进一步表明，算法 B 并非单纯依靠延后任务改善可行性，而是利用空间绕行替代部分长时间等待。需要说明的是，两种算法的邻域规模、单次评价成本和总计算预算并不完全相同，因此表 8、9 主要比较在本文既定求解流程下获得的最终方案质量，不把运行时间差异作为搜索效率优劣的严格证据。

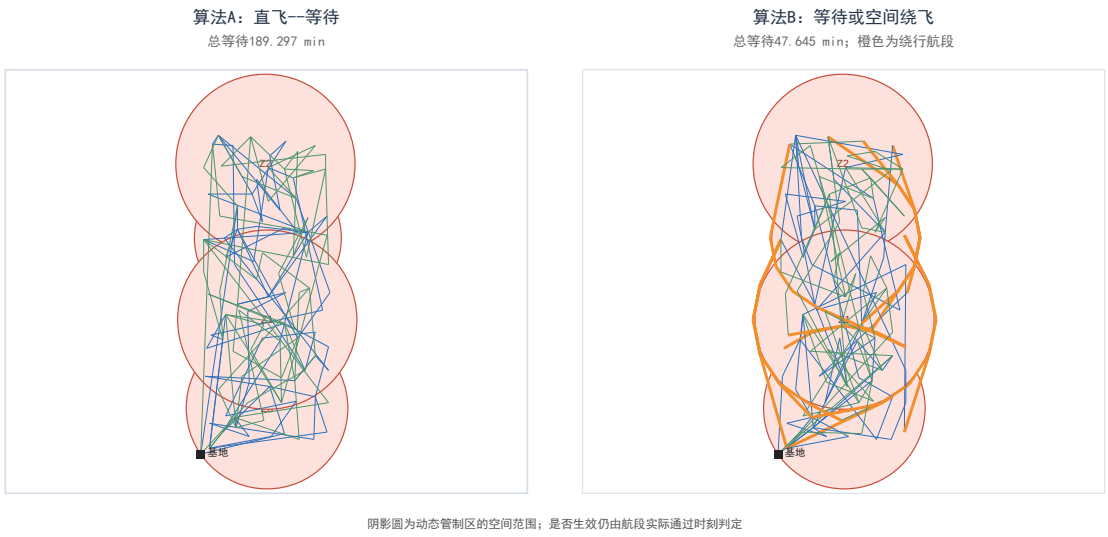


图 5　Case 2 中算法 A 与算法 B 的实际航迹对比（算法 B 橙色折线为绕行组成航段）

图 5 由两套程序的已验证逐事件结果直接绘制。阴影圆表示管制区的空间范围，实际冲突仍由飞行通过时刻与管制时间窗联合判断；因此算法 A 的直线航迹在空间上穿圆并不必然违规，而是在时间冲突发生时等待后再通过。算法 B 则对其中部分冲突航段采用圆外折线，从而显著降低累计等待。

正式方案中，Case 1–Case 4 的最晚返回时刻依次为 16:36:39、21:08:25、17:37:40 和 18:55:29。Case 3 和 Case 4 在均衡阶段主动利用很小的完成时间容差换取更紧凑的工时分布；该取舍已由式 (七.2) 明确限定，不属于任意增加等待。

算法 B 的正式路线、逐事件时间表和 12 组敏感性结果保存于支撑材料 `result3.xlsx`，可复算的结构化结果保存于 `q3_solution.json`；算法 A 的对照结果分别另存为 `result3\_algorithmA.xlsx` 和 `q3\_solution\_algorithmA.json`，避免两套程序和结果混淆。

八　模型检验与敏感性分析

针对前两问，本文采用“求解程序生成结果、独立校验器重新读取原始附件并复算”的方式检查结果一致性。校验器逐条检查基地是否只出现于路线首尾、同一物理点是否连续出现、I/II/III 级巡检点是否分别完成 3/2/1 次有效巡检，并根据

$$T_k = \frac{D_k}{55} + \frac{m_k}{12}$$

重新计算每架无人机的工作时间。其中，D_k 为实际飞行距离，m_k 为有效巡检次数。问题二还额外检查 $N = N^{(1)}$、等待时间为 0 以及 $T^{(2)}_{\max} \le T^{(1)}_{\max} + \varepsilon$。四个算例均通过上述检查，JSON 与 Excel 汇总值和逐路线复算值一致。

问题二主结果取 $\varepsilon = 0$，这是对“保证总体完成时间尽可能短”的保守解释。当 $\varepsilon > 0$

时，可允许极小的总体完成时间损失来换取进一步的均衡改善；但由于当前四个算例已经将极差控制在约 1.43 分钟以内，本文不在主结果中放宽该参数。

8.1 问题三独立可行性检验

问题三另由独立程序重新读取原始附件，并逐事件复核唯一任务副本、基地闭合、时间连续性、每条折线航段的穿圆时段、管制区内服务和非基地安全等待，同时核对 $T_{\max}$、$T_{\min}$、δ 及两阶段效率阈值。算法 B 在 4 个算例、每个算例 3 种机队规模下形成的 12 组结果全部通过验证。该结论证明输出方案满足程序所实现的全部约束，但不等同于对全局最优性的证明。算法 B 的逐事件独立验证程序和算法 A 的快速解析校验程序见附录 D。

8.2 问题三机队规模敏感性

为考察题目对无人机数量未作明确限制时的结果变化，本文另取 $N = N_0, N_0 + 1, N_0 + 2$ 运行算法 B。表 10 仅用于敏感性分析，不替代表 7 的固定 N_0 正式方案。

表 10　算法 B 在不同机队规模下的敏感性结果

算例	N	$T^*_{\max}$/h	$T_{\max}$/h	δ/h	总等待/min	单机最大等待/min
Case 1	4	8.6111	8.6111	0.0174	0.000	0.000
Case 1	5	8.5270	8.5270	0.0615	0.000	0.000
Case 1	6	8.3513	8.3513	0.0732	0.000	0.000
Case 2	2	13.1404	13.1404	0.0003	47.645	34.612
Case 2	3	11.9325	11.9436	0.0081	448.214	233.441
Case 2	4	11.0787	11.0969	0.0108	952.904	269.184
Case 3	5	9.6086	9.6280	0.0458	19.671	19.671
Case 3	6	9.5855	9.6241	0.0778	2.117	2.117
Case 3	7	9.4650	9.4650	0.1435	0.000	0.000
Case 4	4	10.8711	10.9250	0.0652	0.000	0.000
Case 4	5	10.7858	10.7858	0.0169	1.048	1.048
Case 4	6	10.6692	10.6692	0.0183	3.216	3.216

总体上，当前搜索得到的 $T_{\max}$ 随机队规模增加而下降，其中 Case 2 对增加无人机最敏感：由 2 架增加至 3 架和 4 架后，完成时间分别再缩短约 71.8 分钟和 50.8 分钟。表中的总等待是所有无人机等待时间之和，而 $T_{\max}$ 只由最晚返回的一架无人机决定；增机后更多并行路线进入管制时间窗，等待分散在多架无人机上，因而总等待增加并不必然导致系统完成时间增加。Case 2 的单机最大等待虽由 34.612 分钟增至 233.441 分钟和 269.184 分钟，但并行作业仍使最后返回时刻显著提前。δ 并不随 N 单调变化，这是两阶段效率阈值、任务离散性和启发式搜索共同作用的结果。由于题目没有给出新增无人机成本或数量上限，不能仅凭该表定义无条件的“最优机队规模”，故仍以固定 N_0 的方案作为正式答案。

九　模型评价与推广

9.1　模型优点

(1)　**三问之间具有统一且递进的模型结构**。本文将不同等级巡检点的次数要求展开为相互独立的任务副本，在统一框架下依次研究机队规模、系统完成时间和负载均衡问题。问题三继续沿用前两问的任务副本、路线结构和时间计算方法，仅进一步引入具有生效时段的圆形禁飞区，实现了从静态多无人机路径规划到时空耦合路径规划的自然扩展。

(2)　**动态禁飞约束的处理较为完整**。问题三以当天 8:00 为统一时间原点，根据无人机到达各节点的绝对时刻，判断航段飞行、圆内巡检和原地等待是否与管制时间发生冲突。模型不仅检查巡检点是否位于禁飞区内，还对每条航段进行线段与圆形区域的相交计算，避免只检查航段端点而遗漏中途穿越禁飞区的情况。对于 Case 4 中的异常记录 Z8，统一按照半开区间 $[17{:}00, 17{:}00) = \varnothing$ 处理，使数据异常的解释明确且可复现。

(3)　**优化目标的优先级与题意一致**。本文首先在给定机队规模下最小化系统完成时间 $T_{\max}$，再在 $T_{\max} \le (1+\varepsilon_T)T^*_{\max}$ 的限制下最小化工作时间极差 $\delta = T_{\max} - T_{\min}$。这种两阶段方法避免了简单加权和中权重难以解释的问题，也限制了为追求负载均衡而产生的效率损失。算法 A 采用直飞与安全等待相结合的方式，适用于管制稀疏或持续时间较短的情形；算法 B 进一步比较空间绕飞与时间等待，适用于绕飞收益较明显的管制情形。

(4)　**模型结果能够独立复核**。独立校验器从附件原始数据重新读取巡检需求和管制信息，逐架无人机、逐个任务和逐条航段复算任务覆盖次数、路线连续性、飞行时间、巡检时间及禁飞约束。因此，论文模型、程序结果和输出文件之间可以相互核对。需要说明的是，校验器能够确认方案满足约束，但不能据此证明方案达到全局最优。

9.2　模型不足与改进

(1)　**有限计算结果尚不能代替全局最优性证明**。本文得到的是给定算法、参数和计算时间内的最好可行方案。以 Case 1 使用 3 架无人机为例，12 个随机起点并行运行约 6 小时后，最好结果为 $T_{\max} = 10.3195$ h，其余结果也大多集中在 $10.32 \sim 10.50$ h。这说明现有算法容易收敛到相似的局部最优区域，也表明将结果继续压缩至 9 小时具有较大困难。但是，这些搜索过程使用相近的构造方法和邻域算子，不能被视为相互独立的随机试验。因此，6 小时内没有找到可行解只能作为“3 架完成任务较为困难”的计算证据，不能严格证明 3 架无人机不可行，论文中仍应保留 $3 \le N_{\min} \le 4$ 的结论。

(2) **圆形障碍绕行仍存在离散近似**。算法 B 取 $n_b = 16$，相邻边界采样点的圆周角为 $360°/16 = 22.5°$，是在可见图规模和绕行精度之间作出的折中。虽然外偏半径能够保证所构造的离散绕行边不进入原禁飞圆，但连续空间中的最短绕行路径仍被有限个边界节点近似。增加 n_b 可以提高空间分辨率，同时也会增加可见图的节点数、候选边数和时间依赖最短路的计算量。后续可比较 $n_b \in \{8, 16, 32\}$ 时的完成时间、总航程和运行时间，或使用解析切线、圆弧连接及局部自适应加密方法减小离散误差[8]。

(3) **负载均衡参数和机队规模仍带有工程取舍**。本文取 $\varepsilon_T = 0.005$，即允许第二阶段方案的最大完成时间相对第一阶段最优值增加不超过 0.5%。该参数保证了效率优先，但其具体取值仍可通过 $\varepsilon_T \in \{0, 0.001, 0.0025, 0.005\}$ 的敏感性试验进一步验证。与此同时，极差 δ 只由工作时间最长和最短的两架无人机决定，不能完整描述其余无人机的负载分布，因此可补充报告标准差或变异系数，但不再将其加入主优化目标。此外，如果允许无人机数量无限增加而不考虑使用成本，单纯最小化 $T_{\max}$ 会使模型倾向于不断扩充机队，后续可通过边际收益曲线或增加机队投入成本确定更加合理的无人机数量。

(4) **算法仍有进一步改进空间**。针对现有搜索容易停留在 10.3 h 附近的问题，可将目标调整为优先减少超过 9 小时的部分，并从当前最好结果开始逐步压缩时间门槛。局部搜索方面，可采用 ALNS 重点破坏瓶颈路线，并加入 2-opt*、Or-opt、Cross-exchange 和 Ejection chain 等跨路线邻域[9]；同时利用混合遗传搜索维持解的多样性，在交叉和变异后再进行任务修复与局部强化[10-11]。并行计算也可由相互独立的随机起点改为异构岛模型，使不同进程分别运行 ALNS、模拟退火和混合遗传搜索，并通过精英解池交换优质方案。

如果需要严格判定 3 架无人机能否在 9 小时内完成任务，则还需借助精确算法。一种可行思路是先生成工作时间不超过 9 小时的单机路线，再建立集合划分模型，判断能否选择 3 条路线完整覆盖全部任务副本，并进一步采用列生成和分支定价方法闭合上下界[12-13]。启发式算法负责尽可能改进可行解上界，精确算法负责计算下界或给出不可行性证明，两者结合才能更可靠地判断最小机队规模。

(5) **模型对实际飞行环境仍作了确定性简化**。本文假设无人机速度恒定、管制信息在任务开始前完全已知，并忽略电量消耗、风速变化、通信延迟和无人机之间的空中冲突。若用于实际巡检，还需要加入续航与返航电量约束，并在管制区域临时新增、提前结束或延长时采用滚动时域方法，根据无人机当前位置和剩余任务重新规划路径[14]。

参考文献

[1] GUO Y, REN Z, WANG C. iMTSP: solving min-max multiple traveling salesman problem with imperative learning[C/OL]//2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2024: 10245-10252. DOI: 10.1109/IROS 58592.2024.10802039.

[2] BENT R, VAN HENTENRYCK P. A two-stage hybrid local search for the vehicle routing problem with time windows[J/OL]. Transportation Science, 2004, 38(4): 515-530. DOI: 10.1287/trsc.1030.0049.

[3] MOUTHUY S, MASSEN F, DEVILLE Y, et al. A multistage very large-scale neighborhood search for the vehicle routing problem with soft time windows[J/OL]. Transportation Science, 2015, 49(2): 223-238. DOI: 10.1287/trsc.2014.0558.

[4] MATL P, HARTL R F, VIDAL T. Workload equity in vehicle routing problems: a survey and analysis[J/OL]. Transportation Science, 2018, 52(2): 239-260. DOI: 10.1287/trsc.2017.0744.

[5] LEHUÉDÉ F, PÉTON O, TRICOIRE F. A lexicographic minimax approach to the vehicle routing problem with route balancing[J/OL]. European Journal of Operational Research, 2020, 282(1): 129-147. DOI: 10.1016/j.ejor.2019.09.010.

[6] MAVROTAS G. Effective implementation of the epsilon-constraint method in multi-objective mathematical programming problems[J/OL]. Applied Mathematics and Computation, 2009, 213(2): 455-465. DOI: 10.1016/j.amc.2009.03.037.

[7] GENDREAU M, GHIANI G, GUERRIERO E. Time-dependent routing problems: a review[J/OL]. Computers & Operations Research, 2015, 64: 189-197. DOI: 10.101 6/j.cor.2015.06.001.

[8] DEBNATH D, VANEGAS F, BOITEAU S, et al. An integrated geometric obstacle avoidance and genetic algorithm TSP model for UAV path planning[J/OL]. Drones, 2024, 8(7): 302. DOI: 10.3390/drones8070302.

[9] ROPKE S, PISINGER D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows[J/OL]. Transportation Science, 2006, 40(4): 455-472. DOI: 10.1287/trsc.1050.0135.

[10] VIDAL T, CRAINIC T G, GENDREAU M, et al. A hybrid genetic algorithm for multidepot and periodic vehicle routing problems[J/OL]. Operations Research, 2012, 60(3): 611-624. DOI: 10.1287/opre.1120.1048.

[11] VIDAL T. Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood[J/OL]. Computers & Operations Research, 2022, 140: 105643. DOI: 10.1016/j.cor.2021.105643.

[12] FEILLET D. A tutorial on column generation and branch-and-price for vehicle routing problems[J/OL]. 4OR, 2010, 8(4): 407-424. DOI: 10.1007/s10288-010-0130-z.

[13] COSTA L, CONTARDO C, DESAULNIERS G. Exact branch-price-and-cut algorithms for vehicle routing[J/OL]. Transportation Science, 2019, 53(4): 946-985. DOI: 10.1287/trsc.2018.0878.

[14] ADAMO T, GENDREAU M, GHIANI G, et al. A review of recent advances in time-dependent vehicle routing[J/OL]. European Journal of Operational Research, 2024, 319(1): 1-15. DOI: 10.1016/j.ejor.2024.06.016.

人工智能工具使用说明

本队与 AI 工具的交互以“在队员已有思路和程序基础上进行辅助整理、检查与润色”为主。典型提示词及完整交互记录已存档，详见附录文件。
对 AI 输出的采纳、人工修改和核验情况如下：

1. 所有 AI 输出**仅作为辅助参考**，核心决策（变量定义、约束逻辑、算法参数、停止条件、结果判定）均由队员根据实际程序和题目条件独立确定。

2. 对表述类输出：逐句比对后进行大幅删改、重写或补充，确保与程序和计算结果完全一致。

3. 对代码类输出：仅用于解释、补充注释或调试建议，最终程序全部由队员实际编写、运行并独立验证。

4. 对图表与表格：AI 生成初稿或代码后，队员重新计算数值、调整图例与单位、核对正文引用。

5. 语言润色部分：仅采纳符合原意的修改，其余全部人工重写。

6. 最终论文、程序、结果文件及支撑材料均经全体队员人工审阅、复算并共同确认，对原创性、真实性与准确性负全部责任。

郑重声明：本参赛队确保核心建模与分析由队员主导，AI 工具仅用于上述辅助环节。所有 AI 参与完成的内容均已逐项人工审查与核实。

附录 A　支撑材料文件列表

支撑材料以 ZIP 压缩包单独提交。为便于评阅和复核，表 11仅列出文件类别、核心入口及用途；完整目录结构、运行命令、依赖版本和随机参数见压缩包根目录的 `README.md`。赛题提供的原始附件不重复列入自主数据资料。

表 11　支撑材料文件列表

序号	文件或目录	内容及用途
1	`README.md`、各问题 `README.md`	总体目录、运行环境、运行顺序、输入输出与结果复现说明。
2	`requirements.txt`、`environment.yml`、`config.json`	Python 依赖、Conda 环境及问题一搜索参数。
3	`question1/program/run.py`	问题一统一运行入口：机队规模搜索、路线优化、结果输出与复核。
4	`question1/program/src/uav_q1/`	问题一启发式求解、精确模型、独立校验及 Excel 输出模块；完整代码见附录 B。
5	`question1/results/`	问题一正式工作簿、结构化路线和搜索记录。
6	`question2/program/q2_solver.py`	问题二负载均衡求解与独立校验程序；完整代码见附录 C。
7	`question2/results/`	问题二工作簿、结构化方案、汇总指标和逐架路线。
8	`question3/program/q3_solver.py`	问题三算法 A：直飞、必要等待与路线优化。
9	`question3/program/q3_solver_enhanced.py`	问题三算法 B：时变可见图、空间绕飞与两阶段优化。
10	`question3/program/q3_fast_validator.py`、`q3_enhanced_validator.py`	算法 A 快速校验与算法 B 逐事件独立复核；完整代码见附录 D。
11	`question3/results/`	算法 A 对照结果、算法 B 正式结果及机队规模敏感性结果。
12	`generated_figures/`、各问题辅助绘图与工作簿构建程序	论文图件、结果整理和格式转换文件，仅作为电子支撑材料提交。
13	`AI_tool_usage_details.pdf`	人工智能工具名称、用途、人工修改和核验情况。

结果口径：问题三正式方案采用算法 B，固定问题一最终采用的机队规模 $N_0 = (4, 2, 5, 4)$；算法 A 仅作基准对照，$N_0 + 1$、$N_0 + 2$ 仅用于敏感性分析。全部正式结果均由独立程序复核。验证通过仅说明方案满足本文实现的约束，不代表已经证明全局最优。

附录 B　问题一源程序

以下五个文件共同构成问题一完整程序，运行入口为 run.py。测试脚本、环境安装脚本、说明文档和预计算结果仅放入电子支撑材料。

问题一统一运行入口

路径： submission_package/question1/program/run.py

```

from __future__ import annotations

import argparse
import json
import math
import os
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
sys.path.insert(0, str(ROOT / "src"))

# 多进程随机起点优先，避免每个工作进程再启动一组 BLAS/OpenMP 线程。
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")

import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment
from scipy.sparse.csgraph import minimum_spanning_tree

from uav_q1.exact_milp import result_as_dict, solve_fixed_n_exact
from uav_q1.excel_output import export_workbook
from uav_q1.solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM, dist, rtime, solve
from uav_q1.validation import validate_all


def load_config() -> dict:
    return json.loads((ROOT / "config.json").read_text(encoding="utf-8"))


def pack_case(case_name: str, df: pd.DataFrame, routes: list[list[int]]) -> dict:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    packed = []
    for uid, route in enumerate(routes, 1):
        point_ids = [int(df.iloc[i].Point_ID) for i in route]
        packed.append({
            "uav": uid,
            "sequence": [0, *point_ids, 0],
            "distance_km": dist(route, points) * UNIT_KM,
            "service_count": len(route),
            "time_h": rtime(route, points),
        })
    packed.sort(key=lambda x: x["time_h"], reverse=True)
    for uid, route in enumerate(packed, 1):
        route["uav"] = uid
    return {
        "N": len(packed),
        "Tmax": max(x["time_h"] for x in packed),
        "Tmin": min(x["time_h"] for x in packed),
        "routes": packed,
    }


def mst_lower_bound(df: pd.DataFrame, limit_h: float) -> int:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    visits = df.Inspection_Level.map(REQ).to_numpy(int)
    augmented = np.vstack([[0.0, 0.0], points])
    matrix = np.linalg.norm(augmented[:, None, :] - augmented[None, :, :], axis=2) * UNIT_KM
    mst_km = float(minimum_spanning_tree(matrix).sum())
    total_lower_h = visits.sum() * SERVICE_H + mst_km / SPEED
    return max(1, math.ceil(total_lower_h / limit_h - 1e-12))


def _cycle_cover_lower_bound_km(df: pd.DataFrame, n_uav: int) -> float:
    """ 任务副本循环覆盖松弛；同物理点副本间以及基地副本间禁止直连。"""
    physical = np.repeat(
        np.arange(len(df), dtype=int),
        df.Inspection_Level.map(REQ).to_numpy(int),
    )
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    task_xy = points[physical]
    m = len(physical)
    size = m + int(n_uav)
    large = 1.0e12
    cost = np.full((size, size), large, dtype=float)
    task_cost = np.linalg.norm(task_xy[:, None, :] - task_xy[None, :, :], axis=2) * UNIT_KM
    task_cost[physical[:, None] == physical[None, :]] = large
    cost[:m, :m] = task_cost
    depot = np.linalg.norm(task_xy, axis=1) * UNIT_KM
    cost[:m, m:] = depot[:, None]
    cost[m:, :m] = depot[None, :]
    row, col = linear_sum_assignment(cost)
    chosen = cost[row, col]
    return math.inf if np.any(chosen >= large / 2) else float(chosen.sum())


def strong_lower_bound(df: pd.DataFrame, limit_h: float) -> dict:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    demand = df.Inspection_Level.map(REQ).to_numpy(int)
    augmented = np.vstack([[0.0, 0.0], points])
    matrix_units = np.linalg.norm(augmented[:, None, :] - augmented[None, :, :], axis=2)
    mst_km = float(minimum_spanning_tree(matrix_units).sum()) * UNIT_KM
    repeated_degree_units = 0.0
    for idx, count in enumerate(demand, start=1):
        incident = np.delete(matrix_units[idx], idx)
        repeated_degree_units += int(count) * float(np.partition(incident, 1)[:2].sum())
    repeated_degree_km = repeated_degree_units * UNIT_KM / 2.0
    service_h = float(demand.sum()) * SERVICE_H
    farthest_h = float(np.max(2.0 * matrix_units[0, 1:] * UNIT_KM / SPEED + SERVICE_H))
    n = max(1, math.ceil(service_h / limit_h - 1e-12))
    while True:
        cycle_km = _cycle_cover_lower_bound_km(df, n)
        routing_km = max(mst_km, repeated_degree_km, cycle_km)
        aggregate_h = service_h + routing_km / SPEED
        makespan_lb = max(aggregate_h / n, farthest_h)
        if makespan_lb <= limit_h + 1e-12:
            break
        n += 1
    return {
        "N": int(n),
        "service_hours": service_h,
        "mst_km": mst_km,
        "repeated_degree_km": repeated_degree_km,
        "cycle_cover_km": cycle_km,
        "aggregate_work_hours": aggregate_h,
        "makespan_lower_bound_at_N": makespan_lb,
        "farthest_roundtrip_lower_bound_hours": farthest_h,
    }


```

```python
def routes_from_precomputed(df: pd.DataFrame, case: dict) -> list[list[int]]:
    id_to_index = {int(row.Point_ID): i for i, row in df.iterrows()}
    routes = []
    for route in case["routes"]:
        routes.append([id_to_index[int(point_id)] for point_id in route["sequence"] if int(point_id) != 0])
    return routes


def solve_from_scratch(input_path: Path, quality: str, engine: str, config: dict) -> dict:
    book = pd.ExcelFile(input_path)
    results = {}
    seeds = int(config["quality_seeds"][quality])
    for case_name in book.sheet_names:
        df = pd.read_excel(book, sheet_name=case_name)
        lower = mst_lower_bound(df, float(config["maximum_work_hours"]))
        n = lower
        limit_h = float(config["maximum_work_hours"])
        max_n = lower + int(config.get("maximum_n_increment", 6))
        history = []
        all_smaller_proven_infeasible = True
        print(f"[{case_name}] strict search starts at lower bound N={n}; quality={quality}; engine={engine}", flush=True)
        while n <= max_n:
            heuristic_routes = None
            heuristic_obj = None
            if engine in {"heuristic", "hybrid"}:
                (heuristic_obj, heuristic_routes), _, _ = solve(case_name, df, n, seeds=seeds)
                print(f"[{case_name}] N={n}, heuristic best Tmax={heuristic_obj:.6f} h", flush=True)
                record = {
                    "N": n,
                    "heuristic_best_Tmax": float(heuristic_obj),
                    "heuristic_feasible": bool(heuristic_obj <= limit_h + 1e-9),
                }
                history.append(record)
                if heuristic_obj <= limit_h + 1e-9:
                    case = pack_case(case_name, df, heuristic_routes)
                    case["theoretical_lower_bound_N"] = lower
                    case["fleet_size_status"] = (
                        "PROVEN_MINIMUM" if all_smaller_proven_infeasible else "FEASIBLE_CANDIDATE"
                    )
                    case["makespan_status"] = "BEST_FOUND_NOT_PROVEN_OPTIMAL"
                    case["search_history"] = history
                    results[case_name] = case
                    break

            if engine in {"exact", "hybrid"}:
                exact = solve_fixed_n_exact(
                    df,
                    n,
                    limit_h=limit_h,
                    time_limit_s=float(config.get("exact_time_limit_seconds", 300)),
                    mip_rel_gap=float(config.get("exact_mip_relative_gap", 0.0)),
                )
                exact_record = result_as_dict(exact)
                exact_record["N"] = n
                if history and history[-1].get("N") == n:
                    history[-1]["exact"] = exact_record
                else:
                    history.append({"N": n, "exact": exact_record})
                print(f"[{case_name}] N={n}, exact status={exact.status}, objective={exact.objective_h}", flush=True)
                if exact.status in {"OPTIMAL", "FEASIBLE"} and exact.routes is not None:
                    case = pack_case(case_name, df, exact.routes)
                    case["theoretical_lower_bound_N"] = lower
                    case["fleet_size_status"] = (
                        "PROVEN_MINIMUM" if all_smaller_proven_infeasible else "FEASIBLE_CANDIDATE"
                    )
                    case["makespan_status"] = (
                        "PROVEN_OPTIMAL" if exact.status == "OPTIMAL" else "FEASIBLE_NOT_PROVEN_OPTIMAL"
                    )
                    case["search_history"] = history
                    results[case_name] = case
                    break
                if exact.status != "INFEASIBLE":
                    # UNKNOWN 不能当作不可行；继续向上寻找只会得到可行上界。
                    all_smaller_proven_infeasible = False
                else:
                    # 启发式失败也不能当作不可行证明。
                    all_smaller_proven_infeasible = False
            n += 1
        else:
            raise RuntimeError(
                f"{case_name}: 在 N={lower}..{max_n} 内没有找到可行方案; "
                " 请提高搜索质量、精确求解时限或 maximum_n_increment"
            )
    return results


def run_ten_minute_plan(
    input_path: Path,
    config: dict,
    total_seconds: float,
    workers: int,
    candidate_k: int,
    resume_from: Path | None = None,
) -> tuple[dict, dict]:
    """ 在统一墙钟预算内验证热启动并并行改善四个固定车队规模。"""
    started = time.monotonic()
    deadline = started + float(total_seconds)
    reserve = min(float(config.get("ten_minute_save_reserve_seconds", 45)), total_seconds * 0.2)
    precomputed = json.loads((ROOT / "data" / "precomputed_solution.json").read_text(encoding="utf-8"))
    validate_all(input_path, precomputed, config)
    if resume_from is not None:
        resumed = json.loads(resume_from.read_text(encoding="utf-8"))
        validate_all(input_path, resumed, config)
        precomputed = resumed
        print(f" 续搜方案独立校验通过: {resume_from}", flush=True)
    else:
        print("已有 4,2,5,4 方案独立校验通过; 开始强下界与并行改进。", flush=True)

    book = pd.ExcelFile(input_path)
    weights = {k: float(v) for k, v in config.get("ten_minute_case_weights", {}).items()}
    process_order = [name for name in ["Case2", "Case4", "Case1", "Case3"] if name in book.sheet_names]
    improved: dict[str, dict] = {}
    report = {
        "mode": "ten-minute",
        "requested_total_seconds": float(total_seconds),
        "workers": int(workers),
        "candidate_neighborhood_size": int(candidate_k),
        "gpu_used": False,
        "cases": {},
    }

    for position, case_name in enumerate(process_order):
        df = pd.read_excel(book, sheet_name=case_name)
        lower = strong_lower_bound(df, float(config["maximum_work_hours"]))
        initial = precomputed[case_name]
        n_uav = int(initial["N"])
        if n_uav < int(lower["N"]):
            raise AssertionError(f"{case_name}: 已有 N={n_uav} 小于严格下界 {lower['N']}")
        warm_routes = routes_from_precomputed(df, initial)
        prior_history = list(initial.get("search_history", []))
        original_warm_tmax = float(initial.get("warm_start_Tmax", initial["Tmax"]))
```

```

        now = time.monotonic()
        usable = max(1.0, deadline - now - reserve)
        remaining_names = process_order[position:]
        denominator = sum(weights.get(name, 1.0) for name in
        ↪ remaining_names)
        budget = max(1.0, usable * weights.get(case_name, 1.0) /
        ↪ max(denominator, 1e-9))
        status = "PROVEN_MINIMUM" if n_uav == int(lower["N"]) else
        ↪ "FEASIBLE_CANDIDATE"
        print(
            f"[{case_name}] strict lower bound={lower['N']}, fixed
            ↪ N={n_uav}, "
            f"fleet status={status}, search budget={budget:.1f}s,
            ↪ workers={workers}",
            flush=True,
        )
        case_started = time.monotonic()
        (obj, routes), _, _, history = solve(
            case_name,
            df,
            n_uav,
            seeds=4096,
            time_limit_s=budget,
            workers=workers,
            warm_routes=warm_routes,
            candidate_k=candidate_k,
            progress=True,
            return_history=True,
        )
        packed = pack_case(case_name, df, routes)
        packed["theoretical_lower_bound_N"] = int(lower["N"])
        packed["lower_bound_details"] = lower
        packed["fleet_size_status"] = status
        packed["makespan_status"] = "BEST_FOUND_NOT_PROVEN_OPTIMAL"
        packed["search_history"] = prior_history + history
        packed["warm_start_Tmax"] = original_warm_tmax
        packed["improvement_hours"] = float(original_warm_tmax -
        ↪ packed["Tmax"])
        improved[case_name] = packed
        report["cases"][case_name] = {
            "N": n_uav,
            "lower_bound": lower,
            "fleet_size_status": status,
            "budget_seconds": budget,
            "actual_seconds": time.monotonic() - case_started,
            "warm_start_Tmax": original_warm_tmax,
            "resume_input_Tmax": float(initial["Tmax"]),
            "best_Tmax": float(packed["Tmax"]),
            "improvement_hours": float(original_warm_tmax -
            ↪ packed["Tmax"]),
            "completed_starts_this_run": len([x for x in history if
            ↪ x.get("seed") is not None]),
            "completed_starts": len([x for x in prior_history +
            ↪ history if x.get("seed") is not None]),
        }

    ordered = {name: improved[name] for name in book.sheet_names}
    validate_all(input_path, ordered, config)
    report["elapsed_seconds"] = time.monotonic() - started
    report["validation"] = "PASSED"
    return ordered, report


def main() -> None:
    parser = argparse.ArgumentParser(description="多无人机协同巡检问
    ↪ 题一求解器")
    parser.add_argument("--mode", choices=["reproduce", "solve",
    ↪ "ten-minute"], default="reproduce",
                        help="reproduce 复现; solve 完整搜索;
                        ↪ ten-minute 按统一短时预算并行改进")
    parser.add_argument("--quality", choices=["fast", "standard",
    ↪ "thorough"], default="standard")
    parser.add_argument("--engine", choices=["heuristic", "hybrid",
    ↪ "exact"], default="hybrid",
                        help="heuristic 仅搜索; hybrid
                        ↪ 搜索失败后精确判定; exact 仅精确 MILP")
    parser.add_argument("--total-time-limit", type=float,
    ↪ default=None, metavar="SECONDS",
                        help="ten-minute 模式的统一墙钟预算, 默认读取
                        ↪ config.json")
    parser.add_argument("--workers", type=int, default=None,
                        help="并行随机起点进程数, 默认读取
                        ↪ config.json")
    parser.add_argument("--candidate-k", type=int, default=None,
                        help=" 候选近邻数量, 默认读取 config.json")
    parser.add_argument("--input", type=Path, default=ROOT / "input"
    ↪ / "附件1.xlsx")
    parser.add_argument("--output-dir", type=Path, default=None)
    parser.add_argument("--resume-from", type=Path, default=None,
                        help="ten-minute 模式从已校验的 solution.json
                        ↪ 继续热启动搜索")
    args = parser.parse_args()

    config = load_config()
    report = None
    if args.mode == "reproduce":
        results = json.loads((ROOT / "data" /
        ↪ "precomputed_solution.json").read_text(encoding="utf-8"))
    elif args.mode == "solve":
        results = solve_from_scratch(args.input, args.quality,
        ↪ args.engine, config)
    else:
        total_seconds = float(
            args.total_time_limit
            if args.total_time_limit is not None
            else config.get("ten_minute_total_seconds", 600)
        )
        workers = int(args.workers if args.workers is not None else
        ↪ config.get("parallel_workers", 4))
        candidate_k = int(
            args.candidate_k
            if args.candidate_k is not None
            else config.get("candidate_neighborhood_size", 24)
        )
        if total_seconds <= 30 or workers < 1 or candidate_k < 1:
            parser.error("ten-minute 参数要求总时间>30 秒、workers>=1、
            ↪ candidate-k>=1")
        os.environ["LOKY_MAX_CPU_COUNT"] = str(workers)
        results, report = run_ten_minute_plan(
            args.input, config, total_seconds, workers, candidate_k,
            ↪ args.resume_from
        )

    validate_all(args.input, results, config)
    output_dir = args.output_dir or (ROOT / ("outputs_10min" if
    ↪ args.mode == "ten-minute" else "outputs"))
    output_dir.mkdir(parents=True, exist_ok=True)
    json_path = output_dir / "solution.json"
    xlsx_path = output_dir / "result1.xlsx"
    json_path.write_text(json.dumps(results, ensure_ascii=False,
    ↪ indent=2), encoding="utf-8")
    export_workbook(results, xlsx_path, config)
    if report is not None:
        (output_dir / "search_report.json").write_text(
            json.dumps(report, ensure_ascii=False, indent=2),
            ↪ encoding="utf-8"
        )
    print(f"Validation passed.\nJSON: {json_path}\nExcel:
    ↪ {xlsx_path}")


if __name__ == "__main__":
    main()
```

问题一启发式求解核心模块

路径： submission_package/question1/program/src/uav_q1/solver_engine.py

```
import math, json, os, random, time
from concurrent.futures import FIRST_COMPLETED, ProcessPoolExecutor,
↪ wait
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
```

```python
from scipy.sparse.csgraph import minimum_spanning_tree

SPEED=55.0; UNIT_KM=0.1; SERVICE_H=5/60
REQ={'I':3,'II':2,'III':1}

def valid(r):
    return all(r[i]!=r[i+1] for i in range(len(r)-1))

def dist(r,p):
    if not r:return 0.0
    q=p[r]
    return np.linalg.norm(q[0])+np.linalg.norm(q[-1])+np.linalg.norm(q[1:]-q[:-1],axis=1).sum()

def rtime(r,p): return dist(r,p)*UNIT_KM/SPEED+len(r)*SERVICE_H

def distance_matrix(p):
    q=np.vstack([[0.0,0.0],p])
    return np.linalg.norm(q[:,None,:]-q[None,:,:],axis=2)

def rtime_matrix(r,D):
    if not r:return 0.0
    z=[x+1 for x in r]
    units=D[0,z[0]]+D[z[-1],0]
    if len(z)>1:units+=sum(D[a,b] for a,b in zip(z,z[1:]))
    return float(units)*UNIT_KM/SPEED+len(r)*SERVICE_H

def candidate_sets(p,k):
    n=len(p);k=max(1,min(int(k),max(1,n-1)))
    raw=np.linalg.norm(p[:,None,:]-p[None,:,:],axis=2)
    return [set(np.argsort(raw[i])[1:k+1].tolist()) for i in range(n)]

def removal_time(r,pos,current_time,D):
    node=r[pos]+1;left=0 if pos==0 else r[pos-1]+1;right=0 if pos==len(r)-1 else r[pos+1]+1
    delta=D[left,right]-D[left,node]-D[node,right]
    return current_time+float(delta)*UNIT_KM/SPEED-SERVICE_H

def best_insert_fast(r,node,D,near):
    legal=[];preferred=[]
    for pos in range(len(r)+1):
        a=None if pos==0 else r[pos-1];b=None if pos==len(r) else r[pos]
        if a==node or b==node:continue
        ai=0 if a is None else a+1;bi=0 if b is None else b+1;ni=node+1
        delta=float(D[ai,ni]+D[ni,bi]-D[ai,bi])
        item=(delta,pos)
        legal.append(item)
        if a is None or b is None or a in near[node] or b in near[node]:preferred.append(item)
    pool=preferred or legal
    return min(pool) if pool else None

def two_opt(r,p,passes=20):
    r=list(r);n=len(r)
    for _ in range(passes):
        best=0; move=None
        for i in range(n-1):
            a=None if i==0 else r[i-1];b=r[i]
            for j in range(i+1,n):
                c=r[j];d=None if j==n-1 else r[j+1]
                # reversing preserves internal adjacency; check only new boundary pairs
                if a is not None and a==c:continue
                if d is not None and b==d:continue
                old=(np.linalg.norm(p[b]) if a is None else np.linalg.norm(p[a]-p[b]))+(np.linalg.norm(p[c]) if d is None else np.linalg.norm(p[c]-p[d]))
                new=(np.linalg.norm(p[c]) if a is None else np.linalg.norm(p[a]-p[c]))+(np.linalg.norm(p[b]) if d is None else np.linalg.norm(p[b]-p[d]))
                if new-old<best-1e-9:best=new-old;move=(i,j)
        if move is None:break
        i,j=move;r[i:j+1]=reversed(r[i:j+1])
    assert valid(r)
    return r

def insert_options(r,node,p):
    out=[]
    for pos in range(len(r)+1):
        a=None if pos==0 else r[pos-1];b=None if pos==len(r) else r[pos]
        if a==node or b==node:continue
        old=0 if a is None or b is None else np.linalg.norm(p[a]-p[b])
        new=(np.linalg.norm(p[node]) if a is None else np.linalg.norm(p[a]-p[node]))+(np.linalg.norm(p[node]) if b is None else np.linalg.norm(p[node]-p[b]))
        out.append((new-old,pos))
    return out

def can_remove(r,pos):
    return not (0<pos<len(r)-1 and r[pos-1]==r[pos+1])

def init_unique(points,req,k,seed,mode):
    n=len(points);routes=[[] for _ in range(k)]
    if mode=='kmeans':
        labels=KMeans(n_clusters=k,n_init=10,random_state=seed).fit(points,sample_weight=req).labels_
        groups=[np.where(labels==c)[0].tolist() for c in range(k)]
    else:
        ang=np.arctan2(points[:,1],points[:,0]);order=np.argsort(ang).tolist();shift=seed%n;order=order[shift:]+order[:shift]
        groups=[[] for _ in range(k)];loads=[0]*k
        for i in order:
            c=min(range(k),key=lambda z:loads[z]);groups[c].append(i);loads[c]+=req[i]
    for c,ids in enumerate(groups):
        if not ids:continue
        cur=min(ids,key=lambda i:np.linalg.norm(points[i]));routes[c]=[cur];rem=set(ids);rem.remove(cur)
        while rem:
            cur=min(rem,key=lambda j:np.linalg.norm(points[j]-points[cur]));routes[c].append(cur);rem.remove(cur)
        routes[c]=two_opt(routes[c],points)
    return routes

def add_extra_visits(routes,points,req,rng):
    extras=[]
    for i,r in enumerate(req):extras += [i]*(int(r)-1)
    extras.sort(key=lambda i:(-np.linalg.norm(points[i]),rng.random()))
    for node in extras:
        cur=[rtime(r,points) for r in routes];best=None
        for k,r in enumerate(routes):
            for delta,pos in insert_options(r,node,points):
                nt=cur[k]+delta*UNIT_KM/SPEED+SERVICE_H
                obj=max([cur[z] for z in range(len(routes)) if z!=k]+[nt])
                val=(obj,nt,delta)
                if best is None or val<best[0]:best=(val,k,pos)
        if best is None:raise RuntimeError('No valid insertion')
        _,k,pos=best;routes[k].insert(pos,node)
    return [two_opt(r,points) for r in routes]

def improve(routes,p,rng,iters,candidate_k=24):
    routes=[two_opt(r,p) for r in routes];D=distance_matrix(p);near=candidate_sets(p,candidate_k)
    times=np.array([rtime_matrix(r,D) for r in routes])
    best=[r[:] for r in routes];best_obj=times.max();temp=.06
    for it in range(iters):
        src=int(np.argmax(times)) if rng.random()<.72 else rng.randrange(len(routes))
        if not routes[src]:continue
        pos=rng.randrange(len(routes[src]));node=routes[src][pos]
        if not can_remove(routes[src],pos):continue
        nrs=routes[src][:pos]+routes[src][pos+1:];ts=removal_time(routes[src],pos,times[src],D)
        bestmove=None
        for dst in range(len(routes)):
            if dst==src:continue
            choice=best_insert_fast(routes[dst],node,D,near)
            if choice is None:continue
            delta,pos2=choice
            nrd=routes[dst][:pos2]+[node]+routes[dst][pos2:]
            td=times[dst]+delta*UNIT_KM/SPEED+SERVICE_H
            others=[times[z] for z in range(len(routes)) if z not in (src,dst)]
            newmax=max(others+[ts,td]) if others else max(ts,td)
```

```
            cand=(newmax,dst,pos2,nrd,td)
            if bestmove is None or cand[0]<bestmove[0]:bestmove=cand
        if bestmove is None:continue
        newmax,dst,pos2,nrd,td=bestmove
        change=newmax-times.max()
        if change<0 or rng.random()<math.exp(-max(0,change)/max(temp,1e-8)):
            routes[src],routes[dst]=nrs,nrd;times[src],times[dst]=ts,td
            if it%250==0:
                routes[src]=two_opt(routes[src],p,4);routes[dst]=two_opt(routes[dst],p,4)
                times[src]=rtime_matrix(routes[src],D);times[dst]=rtime_matrix(routes[dst],D)
            if times.max()<best_obj-1e-9:
                best_obj=times.max();best=[r[:] for r in routes]
        temp*=.99975
    best=[two_opt(r,p,30) for r in best]
    return polish_balance(best,p,candidate_k=candidate_k)

def polish_balance(routes,p,rounds=80,candidate_k=24):
    routes=[r[:] for r in routes];D=distance_matrix(p);near=candidate_sets(p,candidate_k)
    for _ in range(rounds):
        times=[rtime_matrix(r,D) for r in routes];old=max(times);src=int(np.argmax(times));candidate=None
        # Exhaustive relocate from current longest route.
        for pos,node in enumerate(routes[src]):
            if not can_remove(routes[src],pos):continue
            a=routes[src][:pos]+routes[src][pos+1:];ta=removal_time(routes[src],pos,times[src],D)
            for dst in range(len(routes)):
                if dst==src:continue
                choice=best_insert_fast(routes[dst],node,D,near)
                if choice is None:continue
                delta,q=choice
                b=routes[dst][:q]+[node]+routes[dst][q:];tb=times[dst]+delta*UNIT_KM/SPEED+SERVICE_H
                nm=max([times[z] for z in range(len(routes)) if z not in (src,dst)]+[ta,tb])
                if nm<old-1e-9 and (candidate is None or nm<candidate[0]):candidate=(nm,src,dst,a,b)
        # Exhaustive one-for-one swaps involving current longest route.
        for dst in range(len(routes)):
            if dst==src:continue
            for i,a_node in enumerate(routes[src]):
                for j,b_node in enumerate(routes[dst]):
                    if a_node==b_node:continue
                    if b_node not in near[a_node] and a_node not in near[b_node]:continue
                    a=routes[src][:];b=routes[dst][:];a[i],b[j]=b_node,a_node
                    if not valid(a) or not valid(b):continue
                    ta=rtime_matrix(a,D);tb=rtime_matrix(b,D)
                    nm=max([times[z] for z in range(len(routes)) if z not in (src,dst)]+[ta,tb])
                    if nm<old-1e-9 and (candidate is None or nm<candidate[0]):candidate=(nm,src,dst,a,b)
        if candidate is None:break
        _,src,dst,a,b=candidate
        routes[src]=two_opt(a,p,8);routes[dst]=two_opt(b,p,8)
    return routes

def _progress(label,done,total,best,started,active,time_limit_s=None):
    width=24;elapsed=time.monotonic()-started
    ratio=elapsed/max(.001,time_limit_s) if time_limit_s is not None else done/max(1,total)
    filled=min(width,int(width*ratio));bar='*'*filled+'·'*(width-filled)
    best_text='--' if best is None else f'{best:.5f}h'
    print(f'\r[{label}] [{bar}] {done}/{total} active={active} best={best_text} elapsed={elapsed:6.1f}s',end='',flush=True)

def _solve_one_seed(payload):
    cname,df,k,s,warm_routes,candidate_k=payload
    os.environ.setdefault('OMP_NUM_THREADS','1');os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
    p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
    started=time.monotonic();rng=random.Random(9187+s)
    if warm_routes is not None and s<8:
        routes=[list(r) for r in warm_routes]
    else:
        routes=init_unique(p,req,k,301+s,'kmeans' if s%2==0 else 'sweep')
        routes=add_extra_visits(routes,p,req,rng)
    routes=improve(routes,p,random.Random(40001+s),12000+80*len(df),candidate_k=candidate_k)
    obj=max(rtime(r,p) for r in routes)
    return {'seed':s,'Tmax':float(obj),'routes':routes,'elapsed_seconds':time.monotonic()-started}

def solve(cname,df,k,seeds=14,time_limit_s=None,workers=1,warm_routes=None,
          candidate_k=24,progress=False,return_history=False):
    p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
    started=time.monotonic();history=[];ans=None
    if warm_routes is not None:
        warm=[list(r) for r in warm_routes]
        warm_obj=max(rtime(r,p) for r in warm)
        ans=(warm_obj,warm);history.append({'kind':'warm_start','seed':None,'Tmax':float(warm_obj),'elapsed_seconds':0.0})
    workers=max(1,int(workers));seeds=max(1,int(seeds));deadline=None if time_limit_s is None else started+float(time_limit_s)

    if workers==1:
        for s in range(seeds):
            if deadline is not None and time.monotonic()>=deadline:break
            item=_solve_one_seed((cname,df,k,s,warm_routes,candidate_k));history.append({k:v for k,v in item.items() if k!='routes'})
            if ans is None or item['Tmax']<ans[0]:ans=(item['Tmax'],item['routes'])
            if progress:_progress(f'{cname} N={k}',len(history)-(1 if warm_routes is not None else 0),seeds,ans[0],started,0,time_limit_s)
    else:
        pending={};next_seed=0;done=0
        with ProcessPoolExecutor(max_workers=workers) as pool:
            while next_seed<seeds and len(pending)<workers:
                fut=pool.submit(_solve_one_seed,(cname,df,k,next_seed,warm_routes,candidate_k));pending[fut]=next_seed;next_seed+=1
            while pending:
                completed,_=wait(tuple(pending),timeout=.5,return_when=FIRST_COMPLETED)
                if not completed:
                    if progress:_progress(f'{cname} N={k}',done,seeds,None if ans is None else ans[0],started,len(pending),time_limit_s)
                    continue
                for fut in completed:
                    pending.pop(fut,None);item=fut.result();done+=1
                    history.append({key:value for key,value in item.items() if key!='routes'})
                    if ans is None or item['Tmax']<ans[0]:ans=(item['Tmax'],item['routes'])
                    if (deadline is None or time.monotonic()<deadline) and next_seed<seeds:
                        nxt=pool.submit(_solve_one_seed,(cname,df,k,next_seed,warm_routes,candidate_k));pending[nxt]=next_seed;next_seed+=1
                if progress:_progress(f'{cname} N={k}',done,seeds,None if ans is None else ans[0],started,len(pending),time_limit_s)
                if deadline is not None and time.monotonic()>=deadline:
                    next_seed=seeds
        if progress:print()
    if ans is None:raise RuntimeError(f'{cname} N={k}: no completed seed and no warm start')
```

```
    history.sort(key=lambda x:(x.get('seed') is None,-1 if x.get('seed') is None else x['seed']))
    if return_history:return ans,p,req,history
    return ans,p,req

def main():
    xl=pd.ExcelFile('upload/附件 1.xlsx');out={}
    for cname in xl.sheet_names:
        df=pd.read_excel(xl,sheet_name=cname);p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
        aug=np.vstack([[0,0],p]);D=np.linalg.norm(aug[:,None]-aug[None,:],axis=2)*UNIT_KM
        mst=float(minimum_spanning_tree(D).sum());lb=math.ceil((req.sum()*SERVICE_H+mst/SPEED)/9-1e-12)
        print(cname,'tasks',req.sum(),'service',req.sum()*SERVICE_H,'LB',lb,flush=True)
        k=max(1,lb)
        while True:
            (obj,routes),p,req=solve(cname,df,k)
            times=[rtime(r,p) for r in routes]
            print(' k',k,'T',np.round(sorted(times),5),'valid',all(valid(r) for r in routes),flush=True)
            if max(times)<=9+1e-9:break
            k+=1
        rr=[]
        for uid,r in enumerate(routes,1):
            seq=[int(df.iloc[i].Point_ID) for i in r]
            rr.append({'uav':uid,'sequence':[0]+seq+[0],'distance_km':dist(r,p)*UNIT_KM,'service_count':len(r),'time_h':rtime(r,p)})
        rr.sort(key=lambda x:x['time_h'],reverse=True)
        for uid,x in enumerate(rr,1):x['uav']=uid
        out[cname]={'N':k,'lower_bound_N':lb,'Tmax':max(x['time_h'] for x in rr),'Tmin':min(x['time_h'] for x in rr),'routes':rr}
    json.dump(out,open('q1_results_revisit.json','w'),ensure_ascii=False,indent=2)

if __name__=='__main__':main()
```

问题一固定机队规模精确模型

路径：submission_package/question1/program/src/uav_q1/exact_milp.py

```
""" 固定无人机数量下的精确 MILP 模型。

该模块不替代快速启发式算法，而是用于区分：

* OPTIMAL：已证明固定 N 下的最优最长工作时间；
* INFEASIBLE：已证明固定 N 无法在 9 小时内完成；
* FEASIBLE：时限内得到可行解，但尚未证明最优；
* UNKNOWN：时限内既没有可行解，也没有不可行证明。

模型把每一次巡检展开成一个任务副本。同一物理点的任务副本之间不建立
直接弧，因此同一无人机不能靠原地停留重复计数，但可以在访问其他点后返回。
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import coo_matrix

from .solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM, rtime


@dataclass
class ExactResult:
    status: str
    message: str
    routes: list[list[int]] | None
    objective_h: float | None
    dual_bound_h: float | None
    mip_gap: float | None
    node_count: int | None


class _Rows:
    """ 稀疏线性约束构造器。"""

    def __init__(self) -> None:
        self.row: list[int] = []
        self.col: list[int] = []
        self.data: list[float] = []
        self.lb: list[float] = []
        self.ub: list[float] = []

    def add(self, terms: dict[int, float], lb: float, ub: float) -> None:
        rid = len(self.lb)
        for col, value in terms.items():
            if abs(value) > 1e-15:
                self.row.append(rid)
                self.col.append(col)
                self.data.append(float(value))
        self.lb.append(float(lb))
        self.ub.append(float(ub))

    def constraint(self, nvar: int) -> LinearConstraint:
        matrix = coo_matrix(
            (self.data, (self.row, self.col)),
            shape=(len(self.lb), nvar),
        ).tocsr()
        return LinearConstraint(matrix, np.asarray(self.lb), np.asarray(self.ub))


def _task_copies(df: pd.DataFrame) -> tuple[np.ndarray, np.ndarray]:
    """ 返回每个任务副本对应的物理行号和坐标。"""
    physical: list[int] = []
    for i, level in enumerate(df["Inspection_Level"]):
        physical.extend([i] * REQ[str(level)])
    physical_arr = np.asarray(physical, dtype=int)
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    return physical_arr, points[physical_arr]


def solve_fixed_n_exact(
    df: pd.DataFrame,
    n_uav: int,
    limit_h: float = 9.0,
    time_limit_s: float = 300.0,
    mip_rel_gap: float = 0.0,
) -> ExactResult:
    """ 用完整弧集 MILP 求固定 ``n_uav`` 的 min-max 路径。

    SciPy/HiGHS 返回 status=2 时才记为严格不可行。达到时限且没有解时记为
    UNKNOWN，程序不得据此排除当前 N。
    """
    physical, task_xy = _task_copies(df)
    m = len(physical)
    if n_uav < 1 or n_uav > m:
        raise ValueError("n_uav 必须位于 1 和任务副本总数之间")

    # 节点 0 为基地；1..m 为任务副本。相同物理点的副本之间禁止直接转移。
    xy = np.vstack([[0.0, 0.0], task_xy])
    arcs: list[tuple[int, int]] = []
    for i in range(m + 1):
        for j in range(m + 1):
            if i == j or (i == 0 and j == 0):
                continue
            if i > 0 and j > 0 and physical[i - 1] == physical[j - 1]:
                continue
            arcs.append((i, j))
    a_count = len(arcs)
    out_arcs: list[list[int]] = [[] for _ in range(m + 1)]
    in_arcs: list[list[int]] = [[] for _ in range(m + 1)]
    for a, (i, j) in enumerate(arcs):
        out_arcs[i].append(a)
```

```
        in_arcs[j].append(a)

    # 变量块：[x 二元弧][f 连续单商品流][y 二元分配][Cmax]。
    x0 = 0
    f0 = n_uav * a_count
    y0 = f0 + n_uav * a_count
    c_idx = y0 + n_uav * m
    nvar = c_idx + 1

    def xid(k: int, a: int) -> int:
        return x0 + k * a_count + a

    def fid(k: int, a: int) -> int:
        return f0 + k * a_count + a

    def yid(k: int, i: int) -> int:
        # i 是 1..m 的任务节点号。
        return y0 + k * m + (i - 1)

    c = np.zeros(nvar)
    c[c_idx] = 1.0
    integrality = np.zeros(nvar, dtype=np.uint8)
    integrality[x0:f0] = 1
    integrality[y0:c_idx] = 1
    lower = np.zeros(nvar)
    upper = np.full(nvar, np.inf)
    upper[x0:f0] = 1.0
    upper[f0:y0] = float(m)
    upper[y0:c_idx] = 1.0
    upper[c_idx] = float(limit_h)

    rows = _Rows()

    # 每个任务副本恰由一架无人机完成。
    for i in range(1, m + 1):
        rows.add({yid(k, i): 1.0 for k in range(n_uav)}, 1.0, 1.0)

    # 每架无人机都从基地出发一次并返回一次；任务点入度=出度=分配变量。
    for k in range(n_uav):
        rows.add({xid(k, a): 1.0 for a in out_arcs[0]}, 1.0, 1.0)
        rows.add({xid(k, a): 1.0 for a in in_arcs[0]}, 1.0, 1.0)
        for i in range(1, m + 1):
            out_terms = {xid(k, a): 1.0 for a in out_arcs[i]}
            out_terms[yid(k, i)] = -1.0
            rows.add(out_terms, 0.0, 0.0)
            in_terms = {xid(k, a): 1.0 for a in in_arcs[i]}
            in_terms[yid(k, i)] = -1.0
            rows.add(in_terms, 0.0, 0.0)

    # 单商品流消除不经过基地的子回路。
    for k in range(n_uav):
        depot_terms: dict[int, float] = {}
        for a in out_arcs[0]:
            depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
            ↪  + 1.0
        for a in in_arcs[0]:
            depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
            ↪  - 1.0
        for i in range(1, m + 1):
            depot_terms[yid(k, i)] = -1.0
        rows.add(depot_terms, 0.0, 0.0)

        for i in range(1, m + 1):
            terms: dict[int, float] = {yid(k, i): -1.0}
            for a in in_arcs[i]:
                terms[fid(k, a)] = terms.get(fid(k, a), 0.0) + 1.0
            for a in out_arcs[i]:
                terms[fid(k, a)] = terms.get(fid(k, a), 0.0) - 1.0
            rows.add(terms, 0.0, 0.0)

        for a in range(a_count):
            rows.add({fid(k, a): 1.0, xid(k, a): -float(m)}, -np.inf,
            ↪  0.0)

    arc_time = np.asarray([
        np.linalg.norm(xy[i] - xy[j]) * UNIT_KM / SPEED for i, j in
        ↪  arcs
    ])

    def time_terms(k: int) -> dict[int, float]:
        terms = {xid(k, a): float(arc_time[a]) for a in
        ↪  range(a_count)}
        for i in range(1, m + 1):
            terms[yid(k, i)] = SERVICE_H
        return terms

    # Cmax 上界已经固定为 9 小时；同时以 Cmax 为第二阶段目标。
    all_time_terms = [time_terms(k) for k in range(n_uav)]
    for terms in all_time_terms:
        terms = dict(terms)
        terms[c_idx] = -1.0
        rows.add(terms, -np.inf, 0.0)

    # 对同质无人机按工作时长降序编号，削减车辆置换对称性。
    for k in range(n_uav - 1):
        terms = dict(all_time_terms[k + 1])
        for idx, value in all_time_terms[k].items():
            terms[idx] = terms.get(idx, 0.0) - value
        rows.add(terms, -np.inf, 0.0)

    # 同一物理点的副本无差别：按所分配的无人机编号排序，削减副本对称性。
    for p in np.unique(physical):
        copies = np.flatnonzero(physical == p) + 1
        for left, right in zip(copies[:-1], copies[1:]):
            terms: dict[int, float] = {}
            for k in range(n_uav):
                terms[yid(k, int(left))] = float(k)
                terms[yid(k, int(right))] = -float(k)
            rows.add(terms, -np.inf, 0.0)

    result = milp(
        c=c,
        integrality=integrality,
        bounds=Bounds(lower, upper),
        constraints=rows.constraint(nvar),
        options={
            "time_limit": float(time_limit_s),
            "mip_rel_gap": float(mip_rel_gap),
            "presolve": True,
        },
    )

    status_map = {0: "OPTIMAL", 1: "LIMIT", 2: "INFEASIBLE", 3:
    ↪  "UNBOUNDED", 4: "ERROR"}
    raw_status = status_map.get(int(result.status), "ERROR")
    if raw_status == "INFEASIBLE":
        return ExactResult("INFEASIBLE", result.message, None, None,
                           getattr(result, "mip_dual_bound", None),
                           getattr(result, "mip_gap", None),
                           getattr(result, "mip_node_count", None))
    if result.x is None:
        return ExactResult("UNKNOWN", result.message, None, None,
                           getattr(result, "mip_dual_bound", None),
                           getattr(result, "mip_gap", None),
                           getattr(result, "mip_node_count", None))

    vector = np.asarray(result.x)
    routes: list[list[int]] = []
    for k in range(n_uav):
        route: list[int] = []
        current = 0
        for _ in range(m + 1):
            chosen = [a for a in out_arcs[current] if vector[xid(k,
            ↪  a)] > 0.5]
            if len(chosen) != 1:
                return ExactResult("UNKNOWN", "MILP
                ↪  解的弧结构无法提取为单一路线", None,
                                   None, getattr(result,
                                   ↪  "mip_dual_bound", None),
                                   getattr(result, "mip_gap", None),
                                   getattr(result, "mip_node_count",
                                   ↪  None))
            _, nxt = arcs[chosen[0]]
            if nxt == 0:
                break
            route.append(int(physical[nxt - 1]))
            current = nxt
        else:
            return ExactResult("UNKNOWN", "路线提取超过任务节点数",
            ↪  None, None,
                               getattr(result, "mip_dual_bound",
                               ↪  None),
                               getattr(result, "mip_gap", None),
```

```
                            getattr(result, "mip_node_count",
                            ↪ None))
        routes.append(route)

    physical_points = df[["X_Coordinate",
    ↪ "Y_Coordinate"]].to_numpy(float)
    objective = max(rtime(route, physical_points) for route in
    ↪ routes)
    final_status = "OPTIMAL" if raw_status == "OPTIMAL" else
    ↪ "FEASIBLE"
    return ExactResult(final_status, result.message, routes,
    ↪ objective,
                       getattr(result, "mip_dual_bound", None),
                       getattr(result, "mip_gap", None),
                       getattr(result, "mip_node_count", None))


def result_as_dict(result: ExactResult) -> dict[str, Any]:
    def number(value: Any) -> float | int | None:
        if value is None:
            return None
        if isinstance(value, (int, np.integer)):
            return int(value)
        return float(value)

    return {
        "status": result.status,
        "message": result.message,
        "objective_h": number(result.objective_h),
        "dual_bound_h": number(result.dual_bound_h),
        "mip_gap": number(result.mip_gap),
        "node_count": number(result.node_count),
    }
```

问题一独立结果校验模块

路径：submission_package/question1/program/src/uav_q1/validation.py

```
from __future__ import annotations

from collections import Counter
from pathlib import Path
import math
import pandas as pd


def validate_all(input_path: Path, results: dict, config: dict) ->
↪ None:
    required_map = config["level_required_visits"]
    speed = float(config["speed_kmh"])
    service_h = float(config["service_minutes_per_visit"]) / 60.0
    limit_h = float(config["maximum_work_hours"])
    book = pd.ExcelFile(input_path)
    assert set(results) == set(book.sheet_names), "结果中的 Case
    ↪ 与附件 1 不一致"

    for case_name in book.sheet_names:
        df = pd.read_excel(book, sheet_name=case_name)
        expected = {int(row.Point_ID):
        ↪ int(required_map[row.Inspection_Level]) for _, row in
        ↪ df.iterrows()}
        coordinates = {int(row.Point_ID): (float(row.X_Coordinate),
        ↪ float(row.Y_Coordinate)) for _, row in df.iterrows()}
        found = Counter()
        case = results[case_name]
        assert int(case["N"]) == len(case["routes"]), f"{case_name}:
        ↪ N 与路线行数不符"

        recalculated_times = []
        for route in case["routes"]:
            sequence = [int(x) for x in route["sequence"]]
            assert sequence[0] == sequence[-1] == 0, f"{case_name}:
            ↪ 路线必须以基地 0 为首尾"
            tasks = sequence[1:-1]
            assert all(tasks[i] != tasks[i + 1] for i in
            ↪ range(len(tasks) - 1)), \
                f"{case_name}: 同一点连续出现，未满足离开后返回规则"
            found.update(tasks)
            xy = [(0.0, 0.0), *[coordinates[x] for x in tasks], (0.0,
            ↪ 0.0)]
            coordinate_distance = sum(math.dist(xy[i], xy[i + 1]) for
            ↪ i in range(len(xy) - 1))
            distance_km = coordinate_distance *
            ↪ float(config["coordinate_unit_km"])
            work_h = distance_km / speed + len(tasks) * service_h
            assert abs(work_h - float(route["time_h"])) < 1e-6,
            ↪ f"{case_name}: 工作时间计算不一致"
            assert work_h <= limit_h + 1e-8, f"{case_name}: 存在超过 9
            ↪ h 的路线"
            recalculated_times.append(work_h)

        assert dict(found) == expected, f"{case_name}: 巡检次数不满足
        ↪ I/II/III=3/2/1"
        assert abs(max(recalculated_times) - float(case["Tmax"])) <
        ↪ 1e-6
        assert abs(min(recalculated_times) - float(case["Tmin"])) <
        ↪ 1e-6
```

问题一工作簿输出模块

路径：submission_package/question1/program/src/uav_q1/excel_output.py

```
from __future__ import annotations

from pathlib import Path
from openpyxl import Workbook
from openpyxl.styles import Alignment, Border, Font, PatternFill,
↪ Side
from openpyxl.utils import get_column_letter


NAVY = "17365D"
BLUE = "4472C4"
LIGHT = "D9EAF7"
WHITE = "FFFFFF"
THIN = Side(style="thin", color="D9E2F3")


def style_header(cells) -> None:
    for cell in cells:
        cell.fill = PatternFill("solid", fgColor=BLUE)
        cell.font = Font(bold=True, color=WHITE)
        cell.alignment = Alignment(horizontal="center",
        ↪ vertical="center", wrap_text=True)
        cell.border = Border(left=THIN, right=THIN, top=THIN,
        ↪ bottom=THIN)


def export_workbook(results: dict, output_path: Path, config: dict)
↪ -> None:
    wb = Workbook()
    ws = wb.active
    ws.title = "Summary"
    ws.sheet_view.showGridLines = False
    ws.merge_cells("A1:D1")
    ws["A1"] = " 问题一：多无人机协同巡检结果"
    ws["A1"].fill = PatternFill("solid", fgColor=NAVY)
    ws["A1"].font = Font(bold=True, color=WHITE, size=16)
    ws["A1"].alignment = Alignment(horizontal="center")
    headers = ["测试算例", "无人机数量 N", "最长工作时间 Tmax (h)",
    ↪ "最短工作时间 Tmin (h)"]
    for col, value in enumerate(headers, 1):
        ws.cell(3, col, value)
    style_header(ws[3])
    for row, (case_name, case) in enumerate(results.items(), 4):
        ws.cell(row, 1, case_name)
        ws.cell(row, 2, case["N"])
        ws.cell(row, 3, case["Tmax"])
        ws.cell(row, 4, case["Tmin"])
```

```
        ws.cell(row, 3).number_format = ws.cell(row, 4).number_format = "0.0000"
    ws["A10"] = " 最终计数规则"
    ws["B10"] = "到达并作业 5 min 计 1 次；原地停留不重复计数；离开后返回可再次计数；不同无人机可同时巡检。"
    ws.merge_cells("B10:D11")
    ws["A12"] = " 时间公式"
    ws["B12"] = "T = Flight Distance / 55 + Inspection Count × 5 / 60"
    ws.merge_cells("B12:D12")
    for col, width in zip("ABCD", [20, 25, 30, 30]):
        ws.column_dimensions[col].width = width
    ws.freeze_panes = "A4"

    for case_name, case in results.items():
        sh = wb.create_sheet(case_name)
        sh.sheet_view.showGridLines = False
        max_len = max(len(x["sequence"]) for x in case["routes"])
        headers = ["UAV ID", *[f"{i}th Inspection Point" for i in range(1, max_len + 1)],
                   "Flight Distance (km)", "Inspection Count", "Working Time (h)"]
        for col, value in enumerate(headers, 1):
            sh.cell(5, col, value)
        style_header(sh[5])
        sh["A1"] = f"{case_name} 详细调度方案 (0 表示基地) "
        end_col = get_column_letter(len(headers))
        sh.merge_cells(f"A1:{end_col}1")
        sh["A1"].fill = PatternFill("solid", fgColor=NAVY)
        sh["A1"].font = Font(bold=True, color=WHITE, size=14)
        sh["A1"].alignment = Alignment(horizontal="center")
        sh["A2"] = "N"; sh["B2"] = case["N"]
        sh["C2"] = "Tmax (h)"; sh["D2"] = case["Tmax"]
        sh["E2"] = "Tmin (h)"; sh["F2"] = case["Tmin"]
        for row, route in enumerate(case["routes"], 6):
            values = [route["uav"], *route["sequence"], *([None] * (max_len - len(route["sequence"]))) ,
                      route["distance_km"], route["service_count"]]
            for col, value in enumerate(values, 1): sh.cell(row, col, value)
            distance_col = 2 + max_len
            count_col = distance_col + 1
            time_col = distance_col + 2
            sh.cell(row, time_col,
            f"={get_column_letter(distance_col)}{row}/55+{get_column_letter(count_col)}{row}*5/60")
            sh.cell(row, distance_col).number_format = "0.000"
            sh.cell(row, time_col).number_format = "0.000000"
        sh.freeze_panes = "B6"
        sh.column_dimensions["A"].width = 10
        for c in range(2, 2 + max_len):
            sh.column_dimensions[get_column_letter(c)].width = 11
        for c in range(2 + max_len, len(headers) + 1):
            sh.column_dimensions[get_column_letter(c)].width = 19
        sh.auto_filter.ref = f"A5:{end_col}{5 + len(case['routes'])}"

    output_path.parent.mkdir(parents=True, exist_ok=True)
    wb.save(output_path)
```

附录 C　问题二源程序

问题二求解、结果复算及 CSV 输出均集中在同一程序中；用于生成论文图件和最终 Excel 版式的辅助脚本仅放入电子支撑材料。

问题二负载均衡求解与校验程序

路径：submission_package/question2/program/q2_solver.py

```python
""" 问题二：固定问题一无人机数量与完成时间上界的负载均衡优化。

Python 求解器只输出 JSON/CSV；result2.xlsx 由 build_result2.mjs 根据同一 JSON 生成。
工作时间仅包含实际飞行与有效巡检服务，不允许用等待时间改善均衡指标。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from collections import Counter
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd


SPEED_KMH = 55.0
UNIT_KM = 0.1
SERVICE_H = 5.0 / 60.0
REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-9


@dataclass(frozen=True)
class CaseData:
    point_ids: tuple[int, ...]
    coordinates: dict[int, tuple[float, float]]
    expected_visits: dict[int, int]
    distance_km: dict[tuple[int, int], float]


def load_case(book: pd.ExcelFile, case_name: str) -> CaseData:
    df = pd.read_excel(book, sheet_name=case_name)
    ids = tuple(int(x) for x in df["Point_ID"])
    coordinates = {
        int(row.Point_ID): (float(row.X_Coordinate), float(row.Y_Coordinate))
        for _, row in df.iterrows()
    }
    coordinates[0] = (0.0, 0.0)
    expected = {
        int(row.Point_ID): REQ[str(row.Inspection_Level)]
        for _, row in df.iterrows()
    }
    distance: dict[tuple[int, int], float] = {}
    for i in (0,) + ids:
        xi, yi = coordinates[i]
        for j in (0,) + ids:
            xj, yj = coordinates[j]
            distance[(i, j)] = math.hypot(xi - xj, yi - yj) * UNIT_KM
    return CaseData(ids, coordinates, expected, distance)


def route_valid(route: list[int]) -> bool:
    return bool(route) and all(a != b for a, b in zip(route, route[1:]))


def route_metrics(route: list[int], data: CaseData) -> tuple[float, float]:
    seq = [0] + route + [0]
    distance = sum(data.distance_km[(a, b)] for a, b in zip(seq, seq[1:]))
    return distance, distance / SPEED_KMH + len(route) * SERVICE_H


def all_metrics(routes: list[list[int]], data: CaseData) -> tuple[list[float], list[float]]:
    pairs = [route_metrics(route, data) for route in routes]
    return [x[0] for x in pairs], [x[1] for x in pairs]


def objective(times_h: list[float]) -> tuple[float, float, float]:
    return max(times_h) - min(times_h), max(times_h), sum(times_h)


def better(a: tuple[float, float, float], b: tuple[float, float, float]) -> bool:
    for x, y in zip(a, b):
        if x < y - TOL:
            return True
        if x > y + TOL:
            return False
    return False


def two_opt(route: list[int], data: CaseData, max_passes: int = 2) -> list[int]:
    """ 仅执行缩短飞行距离的合法 2-opt，不会人为延长轻载路线。"""
    best = route[:]
    if len(best) < 4:
        return best
    for _ in range(max_passes):
        changed = False
        for i in range(len(best) - 2):
            a = 0 if i == 0 else best[i - 1]
            b = best[i]
            for j in range(i + 1, len(best)):
                c = best[j]
                d = 0 if j == len(best) - 1 else best[j + 1]
                old = data.distance_km[(a, b)] + data.distance_km[(c, d)]
                new = data.distance_km[(a, c)] + data.distance_km[(b, d)]
                if new >= old - 1.0e-12:
                    continue
                candidate = best[:i] + list(reversed(best[i : j + 1])) + best[j + 1 :]
                if route_valid(candidate):
                    best = candidate
                    changed = True
                    break
            if changed:
                break
        if not changed:
            break
    return best


def normalize_changed(
    routes: list[list[int]], data: CaseData, changed: tuple[int, ...]
) -> list[list[int]]:
    result = [r[:] for r in routes]
    for idx in set(changed):
        result[idx] = two_opt(result[idx], data, max_passes=2)
    return result


def legal_remove(route: list[int], pos: int) -> bool:
```

```python
    if len(route) <= 1:
        return False
    candidate = route[:pos] + route[pos + 1 :]
    return route_valid(candidate)


def legal_insert(route: list[int], node: int, pos: int) -> bool:
    left = None if pos == 0 else route[pos - 1]
    right = None if pos == len(route) else route[pos]
    return left != node and right != node


def choose_heavy_light(times_h: list[float], rng: random.Random) -> tuple[int, int]:
    order = sorted(range(len(times_h)), key=times_h.__getitem__)
    width = min(2, max(1, len(order) // 2))
    if rng.random() < 0.85:
        src = rng.choice(order[-width:])
        dst = rng.choice(order[:width])
    else:
        src, dst = rng.sample(order, 2)
    return src, dst


def propose(
    routes: list[list[int]], data: CaseData, rng: random.Random
) -> list[list[int]] | None:
    _, times = all_metrics(routes, data)
    src, dst = choose_heavy_light(times, rng)
    move = rng.random()
    candidate = [r[:] for r in routes]

    if move < 0.60:
        positions = [i for i in range(len(candidate[src])) if legal_remove(candidate[src], i)]
        if not positions:
            return None
        pos = rng.choice(positions)
        node = candidate[src].pop(pos)
        legal = [q for q in range(len(candidate[dst]) + 1) if legal_insert(candidate[dst], node, q)]
        if not legal:
            return None
        # 在合法位置中优先选择飞行距离增量较小的几个，再保留随机性。
        ranked = []
        for q in legal:
            left = 0 if q == 0 else candidate[dst][q - 1]
            right = 0 if q == len(candidate[dst]) else candidate[dst][q]
            inc = (
                data.distance_km[(left, node)]
                + data.distance_km[(node, right)]
                - data.distance_km[(left, right)]
            )
            ranked.append((inc, q))
        ranked.sort()
        q = rng.choice(ranked[: min(4, len(ranked))])[1]
        candidate[dst].insert(q, node)
        changed = (src, dst)
    else:
        i = rng.randrange(len(candidate[src]))
        j = rng.randrange(len(candidate[dst]))
        candidate[src][i], candidate[dst][j] = candidate[dst][j], candidate[src][i]
        if not route_valid(candidate[src]) or not route_valid(candidate[dst]):
            return None
        changed = (src, dst)

    candidate = normalize_changed(candidate, data, changed)
    if not all(route_valid(r) for r in candidate):
        return None
    return candidate


def anneal_seed(
    initial: list[list[int]],
    data: CaseData,
    cap_h: float,
    seed: int,
    iterations: int,
    deadline: float,
) -> tuple[list[list[int]], dict]:
    rng = random.Random(seed)
    current = [two_opt(r, data, max_passes=20) for r in initial]
    _, current_times = all_metrics(current, data)
    current_key = objective(current_times)
    if current_key[1] > cap_h + TOL:
        # 问题一初始路线必须始终可作为保底方案。
        current = [r[:] for r in initial]
        _, current_times = all_metrics(current, data)
        current_key = objective(current_times)
    best = [r[:] for r in current]
    best_key = current_key
    accepted = 0
    attempted = 0

    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, data, rng)
        if candidate is None:
            continue
        attempted += 1
        _, times = all_metrics(candidate, data)
        key = objective(times)
        if key[1] > cap_h + TOL or key[1] > 9.0 + TOL:
            continue

        accept = better(key, current_key)
        if not accept:
            progress = iteration / max(1, iterations - 1)
            temperature = 0.02 * (1.0 - progress) + 0.00015
            loss = max(0.0, key[0] - current_key[0])
            loss += 0.20 * max(0.0, key[1] - current_key[1])
            loss += 0.002 * max(0.0, key[2] - current_key[2])
            accept = rng.random() < math.exp(-loss / temperature)
        if accept:
            current = candidate
            current_key = key
            accepted += 1
            if better(key, best_key):
                best = [r[:] for r in candidate]
                best_key = key

    return best, {
        "seed": seed,
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "best_delta_h": best_key[0],
        "best_Tmax_h": best_key[1],
        "best_total_h": best_key[2],
    }


def deterministic_polish(
    initial: list[list[int]], data: CaseData, cap_h: float, max_rounds: int = 80
) -> list[list[int]]:
    """ 系统枚举重载到轻载的单任务迁移，直到没有词典序改善。"""
    routes = [r[:] for r in initial]
    for _ in range(max_rounds):
        _, times = all_metrics(routes, data)
        old_key = objective(times)
        order = sorted(range(len(routes)), key=times.__getitem__)
        sources = order[-min(2, len(order)) :]
        targets = order[: min(2, len(order))]
        best_candidate = None
        best_key = old_key
        for src in sources:
            for dst in targets:
                if src == dst:
                    continue
                for i, node in enumerate(routes[src]):
                    if not legal_remove(routes[src], i):
                        continue
                    for q in range(len(routes[dst]) + 1):
                        if not legal_insert(routes[dst], node, q):
                            continue
                        candidate = [r[:] for r in routes]
                        candidate[src].pop(i)
                        candidate[dst].insert(q, node)
```

```
                        candidate = normalize_changed(candidate, data, (src, dst))
                        _, cand_times = all_metrics(candidate, data)
                        key = objective(cand_times)
                        if key[1] <= cap_h + TOL and key[1] <= 9.0 + TOL and better(key, best_key):
                            best_candidate = candidate
                            best_key = key
        if best_candidate is None:
            break
        routes = best_candidate
    # 最终把每条路线充分缩短，避免通过保留可消除的绕行来抬高 Tmin。
    return [two_opt(route, data, max_passes=100) for route in routes]


def validate_case(
    case_name: str,
    routes: list[list[int]],
    data: CaseData,
    q1_case: dict,
    epsilon_h: float,
) -> tuple[list[float], list[float]]:
    assert len(routes) == int(q1_case["N"]), f"{case_name}: 无人机数量发生变化"
    assert all(route_valid(r) for r in routes), f"{case_name}: 存在空路线或连续相同巡检点"
    actual = Counter(x for route in routes for x in route)
    assert actual == Counter(data.expected_visits), f"{case_name}: 巡检次数不一致"
    distances, times = all_metrics(routes, data)
    cap = float(q1_case["Tmax"]) + epsilon_h
    assert max(times) <= cap + 1.0e-8, f"{case_name}: 超过问题一完成时间上界"
    assert max(times) <= 9.0 + 1.0e-8, f"{case_name}: 超过 9 小时"
    return distances, times


def solve_case(
    case_name: str,
    data: CaseData,
    q1_case: dict,
    epsilon_h: float,
    seeds: int,
    iterations: int,
    time_limit_s: float,
) -> dict:
    initial = [list(map(int, route["sequence"][1:-1])) for route in q1_case["routes"]]
    validate_case(case_name, initial, data, q1_case, epsilon_h)
    cap = float(q1_case["Tmax"]) + epsilon_h
    deadline = time.monotonic() + time_limit_s
    best = [r[:] for r in initial]
    _, best_times = all_metrics(best, data)
    best_key = objective(best_times)
    history = []

    for offset in range(seeds):
        if time.monotonic() >= deadline:
            break
        routes, record = anneal_seed(
            initial, data, cap, 20260818 + offset, iterations, deadline
        )
        history.append(record)
        _, times = all_metrics(routes, data)
        key = objective(times)
        if better(key, best_key):
            best, best_key = routes, key

    best = deterministic_polish(best, data, cap)
    distances, times = validate_case(case_name, best, data, q1_case, epsilon_h)
    delta1 = float(q1_case["Tmax"]) - float(q1_case["Tmin"])
    delta2 = max(times) - min(times)
    route_records = []
    for k, (route, distance, work_h) in enumerate(zip(best, distances, times), start=1):
        route_records.append(
            {
                "uav": k,
                "sequence": [0] + route + [0],
                "distance_km": distance,
                "service_count": len(route),
                "time_h": work_h,
                "waiting_time_h": 0.0,
            }
        )
    return {
        "N": int(q1_case["N"]),
        "q1_Tmax_h": float(q1_case["Tmax"]),
        "q1_Tmin_h": float(q1_case["Tmin"]),
        "q1_delta_h": delta1,
        "epsilon_h": epsilon_h,
        "completion_time_cap_h": cap,
        "Tmax": max(times),
        "Tmin": min(times),
        "delta": delta2,
        "total_work_h": sum(times),
        "delta_reduction_h": delta1 - delta2,
        "delta_reduction_pct": 0.0 if delta1 <= TOL else (delta1 - delta2) / delta1,
        "objective_order": ["delta", "Tmax", "total_work_h"],
        "waiting_allowed": False,
        "routes": route_records,
        "search_history": history,
        "validation": "PASSED",
    }


def write_csv_outputs(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary.csv").open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h", "total_work_h"])
        for case_name, result in results.items():
            w.writerow([case_name, result["N"], result["Tmax"], result["Tmin"], result["delta"], result["total_work_h"]])
    with (output_dir / "comparison.csv").open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["Case", "q1_delta_h", "q2_delta_h", "reduction_h", "reduction_pct"])
        for case_name, result in results.items():
            w.writerow([case_name, result["q1_delta_h"], result["delta"], result["delta_reduction_h"], result["delta_reduction_pct"]])
    for case_name, result in results.items():
        max_len = max(len(route["sequence"]) for route in result["routes"])
        headers = ["UAV_ID"] + [f"Stop_{i}" for i in range(1, max_len + 1)] + ["Distance_km", "Service_Count", "Working_Time_h"]
        with (output_dir / f"{case_name}_routes.csv").open("w", newline="", encoding="utf-8-sig") as f:
            w = csv.writer(f)
            w.writerow(headers)
            for route in result["routes"]:
                padding = [""] * (max_len - len(route["sequence"]))
                w.writerow([route["uav"], *route["sequence"], *padding, route["distance_km"], route["service_count"], route["time_h"]])


def validate_saved(input_path: Path, q1_path: Path, q2_path: Path) -> None:
    q1 = json.loads(q1_path.read_text(encoding="utf-8"))
    q2 = json.loads(q2_path.read_text(encoding="utf-8"))
    book = pd.ExcelFile(input_path)
    assert set(q1) == set(q2) == set(book.sheet_names)
    for case_name in book.sheet_names:
        data = load_case(book, case_name)
        routes = [route["sequence"][1:-1] for route in q2[case_name]["routes"]]
        distances, times = validate_case(case_name, routes, data, q1[case_name], q2[case_name]["epsilon_h"])
        assert abs(max(times) - q2[case_name]["Tmax"]) < 1.0e-8
        assert abs(min(times) - q2[case_name]["Tmin"]) < 1.0e-8
        for saved, distance, work_h in zip(q2[case_name]["routes"], distances, times):
```

```
            assert abs(saved["distance_km"] - distance) < 1.0e-8
            assert abs(saved["time_h"] - work_h) < 1.0e-8
            assert saved["waiting_time_h"] == 0.0


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, required=True)
    parser.add_argument("--q1-solution", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--epsilon", type=float, default=0.0)
    parser.add_argument("--seeds", type=int, default=8)
    parser.add_argument("--iterations", type=int, default=30000)
    parser.add_argument("--time-limit-per-case", type=float,
    ↪  default=45.0)
    parser.add_argument("--validate-only", action="store_true")
    args = parser.parse_args()

    output_json = args.output_dir / "q2_solution.json"
    if args.validate_only:
        validate_saved(args.input, args.q1_solution, output_json)
        print(" 问题二保存结果独立校验通过。")
        return

    if args.epsilon < 0:
        raise ValueError("epsilon 必须非负")
    q1 = json.loads(args.q1_solution.read_text(encoding="utf-8"))
    book = pd.ExcelFile(args.input)
    results = {}
    for case_name in book.sheet_names:
        started = time.monotonic()
        data = load_case(book, case_name)
        results[case_name] = solve_case(
            case_name,
            data,
            q1[case_name],
            args.epsilon,
            args.seeds,
            args.iterations,
            args.time_limit_per_case,
        )
        r = results[case_name]
        print(
            f"{case_name}: N={r['N']} Tmax={r['Tmax']:.6f} "
            f"Tmin={r['Tmin']:.6f} delta={r['delta']:.6f} "
            f"elapsed={time.monotonic()-started:.1f}s",
            flush=True,
        )
    args.output_dir.mkdir(parents=True, exist_ok=True)
    output_json.write_text(json.dumps(results, ensure_ascii=False,
    ↪  indent=2), encoding="utf-8")
    write_csv_outputs(results, args.output_dir)
    validate_saved(args.input, args.q1_solution, output_json)
    print(f" 问题二全部校验通过: {output_json}")


if __name__ == "__main__":
    main()
```

附录 D　问题三源程序

以下收入算法 A、算法 B 及两套独立验证程序。算法 B 直接调用算法 A 文件中的数据结构和基础时空冲突函数，因此二者共同构成完整可运行程序。绘图、结果筛选和工作簿格式转换脚本仅放入电子支撑材料。

问题三算法 A 主程序

路径： submission_package/question3/program/q3_solver.py

```python
""" 问题三：动态圆形飞行管制下的多无人机协同巡检优化。

固定问题一/二采用的无人机数量，以问题二路线为初始解；时间以 8:00 为零点、分钟为单位。目标按 (Tmax, delta, total_work) 词典序优化。等待仅在航段或下一巡检服务受管制时发生，不插入用于人为改善均衡性的空等待。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from collections import Counter
from dataclasses import dataclass
from datetime import datetime, timedelta
from pathlib import Path

import pandas as pd


SPEED_KMH = 55.0
UNIT_KM = 0.1
SERVICE_MIN = 5.0
REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-9


@dataclass(frozen=True)
class Zone:
    zone_id: str
    center: tuple[float, float]
    radius: float
    start_min: float
    end_min: float

    @property
    def active(self) -> bool:
        return self.end_min > self.start_min + TOL


@dataclass(frozen=True)
class CaseData:
    coordinates: dict[int, tuple[float, float]]
    expected_visits: dict[int, int]
    zones: tuple[Zone, ...]


def clock_to_min(value: object) -> float:
    text = str(value).strip()
    parsed = datetime.strptime(text, "%H:%M")
    return (parsed.hour - 8) * 60 + parsed.minute


def min_to_clock(value: float) -> str:
    stamp = datetime(2000, 1, 1, 8, 0) + timedelta(minutes=value)
    return stamp.strftime("%H:%M:%S")


def load_cases(points_path: Path, zones_path: Path) -> dict[str, CaseData]:
    point_book = pd.ExcelFile(points_path)
    zone_book = pd.ExcelFile(zones_path)
    assert point_book.sheet_names == zone_book.sheet_names
    result: dict[str, CaseData] = {}
    for case_name in point_book.sheet_names:
        points = pd.read_excel(point_book, sheet_name=case_name)
        zones_df = pd.read_excel(zone_book, sheet_name=case_name)
        coordinates = {0: (0.0, 0.0)}
        expected: dict[int, int] = {}
        for _, row in points.iterrows():
            point_id = int(row.Point_ID)
            coordinates[point_id] = (float(row.X_Coordinate), float(row.Y_Coordinate))
            expected[point_id] = REQ[str(row.Inspection_Level)]
        zones = tuple(
            Zone(
                str(row.Zone_ID),
                (float(row.Center_X), float(row.Center_Y)),
                float(row.Radius),
                clock_to_min(row.Start_Time),
                clock_to_min(row.End_Time),
            )
            for _, row in zones_df.iterrows()
        )
        result[case_name] = CaseData(coordinates, expected, zones)
    return result


def point_inside(point: tuple[float, float], zone: Zone) -> bool:
    return math.dist(point, zone.center) <= zone.radius + 1.0e-10


def segment_circle_interval(
    a: tuple[float, float], b: tuple[float, float], zone: Zone
) -> tuple[float, float] | None:
    """ 返回线段位于闭圆内的参数区间，线段参数 lambda 属于 [0,1]。"""
    dx, dy = b[0] - a[0], b[1] - a[1]
    fx, fy = a[0] - zone.center[0], a[1] - zone.center[1]
    aa = dx * dx + dy * dy
    if aa <= TOL:
        return (0.0, 1.0) if point_inside(a, zone) else None
    bb = 2.0 * (fx * dx + fy * dy)
    cc = fx * fx + fy * fy - zone.radius * zone.radius
    disc = bb * bb - 4.0 * aa * cc
    if disc < -1.0e-10:
        return None
    disc = max(0.0, disc)
    root = math.sqrt(disc)
    lo = max(0.0, (-bb - root) / (2.0 * aa))
    hi = min(1.0, (-bb + root) / (2.0 * aa))
    if lo > hi + 1.0e-10:
        return None
    return lo, hi


def overlaps(a0: float, a1: float, b0: float, b1: float) -> bool:
    return max(a0, b0) < min(a1, b1) - 1.0e-9


def departure_blocks(
    a: tuple[float, float],
    b: tuple[float, float],
    travel_min: float,
    service_at_b: bool,
    zones: tuple[Zone, ...],
) -> list[tuple[float, float, str, str]]:
    blocks: list[tuple[float, float, str, str]] = []
    for zone in zones:
        if not zone.active:
            continue
```

```
        interval = segment_circle_interval(a, b, zone)
        if interval is not None:
            lo, hi = interval
            blocks.append(
                (
                    zone.start_min - hi * travel_min,
                    zone.end_min - lo * travel_min,
                    zone.zone_id,
                    "flight",
                )
            )
        if service_at_b and point_inside(b, zone):
            blocks.append(
                (
                    zone.start_min - travel_min - SERVICE_MIN,
                    zone.end_min - travel_min,
                    zone.zone_id,
                    "service",
                )
            )
    return blocks


def unsafe_handoff_blocks(
    current: tuple[float, float],
    current_travel_min: float,
    outgoing: list[tuple[float, float, str, str]],
    zones: tuple[Zone, ...],
) -> list[tuple[float, float, str, str]]:
    """把“到点服务后不能在管制圆内等待”的约束前移到上一航段出发时刻。"""
    holding_zones = [zone for zone in zones if zone.active and point_inside(current, zone)]
    if not holding_zones:
        return []
    offset = current_travel_min + SERVICE_MIN
    result: list[tuple[float, float, str, str]] = []
    for block_start, block_end, cause_id, _ in outgoing:
        for holding in holding_zones:
            # 若完成服务时刻 r 落入出站禁用区间，等待到 block_end；
            # 只要 [r, block_end) 与当前点所在管制区重叠，
            # 就必须在上一点延迟。
            unsafe_end = min(block_end, holding.end_min)
            if block_end > holding.start_min + TOL and block_start < unsafe_end - TOL:
                result.append(
                    (
                        block_start - offset,
                        unsafe_end - offset,
                        f"{holding.zone_id}/{cause_id}",
                        "handoff",
                    )
                )
    return result


def earliest_departure(
    ready: float, blocks: list[tuple[float, float, str, str]]
) -> tuple[float, list[str]]:
    depart = ready
    reasons: set[str] = set()
    for _ in range(len(blocks) + 2):
        covering = [item for item in blocks if item[0] - TOL <= depart < item[1] - TOL]
        if not covering:
            return depart, sorted(reasons)
        reasons.update(f"{zone_id}:{kind}" for _, _, zone_id, kind in covering)
        depart = max(item[1] for item in covering)
    raise RuntimeError(" 禁用出发区间合并未收敛")


def wait_is_safe(
    node: int, point: tuple[float, float], start: float, end: float, zones: tuple[Zone, ...]
) -> bool:
    if node == 0 or end <= start + TOL:
        return True
    for zone in zones:
        if zone.active and point_inside(point, zone) and overlaps(start, end, zone.start_min, zone.end_min):
            return False
    return True


def schedule_route(
    tasks: list[int], data: CaseData, debug: bool = False, release_min: float = 0.0
) -> dict | None:
    if not tasks or any(a == b for a, b in zip(tasks, tasks[1:])):
        return None
    sequence = [0] + tasks + [0]
    leg_data = []
    effective_blocks: list[list[tuple[float, float, str, str]]] = []
    for a_id, b_id in zip(sequence, sequence[1:]):
        a, b = data.coordinates[a_id], data.coordinates[b_id]
        distance = math.dist(a, b) * UNIT_KM
        travel = distance / SPEED_KMH * 60.0
        leg_data.append((a_id, b_id, a, b, distance, travel))
        effective_blocks.append(departure_blocks(a, b, travel, b_id != 0, data.zones))
    # 从路线末端向前传递：若下一航段会迫使无人机在管制圆内等待，
    # 则把相应禁用时段推回到上一安全节点的出发决策。
    for idx in range(len(leg_data) - 2, -1, -1):
        _, b_id, _, b, _, travel = leg_data[idx]
        if b_id != 0:
            effective_blocks[idx].extend(
                unsafe_handoff_blocks(b, travel, effective_blocks[idx + 1], data.zones)
            )

    now = release_min
    distance_km = 0.0
    wait_min = release_min
    events: list[dict] = []
    if release_min > TOL:
        events.append(
            {
                "type": "wait",
                "node": 0,
                "start_min": 0.0,
                "end_min": release_min,
                "duration_min": release_min,
                "reason_zone_ids": ["safe_release_after_all_zones"],
            }
        )
    for leg, (a_id, b_id, a, b, distance, travel) in enumerate(leg_data, start=1):
        blocks = effective_blocks[leg - 1]
        depart, reasons = earliest_departure(now, blocks)
        if not wait_is_safe(a_id, a, now, depart, data.zones):
            if debug:
                print(
                    f"unsafe wait: node={a_id}, ready={now:.3f}, depart={depart:.3f}, "
                    f"next={b_id}, reasons={reasons}",
                    flush=True,
                )
            return None
        if depart > now + TOL:
            events.append(
                {
                    "type": "wait",
                    "node": a_id,
                    "start_min": now,
                    "end_min": depart,
                    "duration_min": depart - now,
                    "reason_zone_ids": reasons,
                }
            )
            wait_min += depart - now
        arrival = depart + travel
        events.append(
            {
                "type": "flight",
                "leg": leg,
                "from": a_id,
                "to": b_id,
                "start_min": depart,
                "end_min": arrival,
                "duration_min": travel,
                "distance_km": distance,
            }
        )
```

```
        distance_km += distance
        now = arrival
        if b_id != 0:
            events.append(
                {
                    "type": "service",
                    "node": b_id,
                    "start_min": now,
                    "end_min": now + SERVICE_MIN,
                    "duration_min": SERVICE_MIN,
                }
            )
            now += SERVICE_MIN
    return {
        "sequence": sequence,
        "distance_km": distance_km,
        "service_count": len(tasks),
        "flight_time_h": distance_km / SPEED_KMH,
        "wait_time_h": wait_min / 60.0,
        "time_h": now / 60.0,
        "return_clock": min_to_clock(now),
        "events": events,
    }


def valid_tasks(tasks: list[int]) -> bool:
    return bool(tasks) and all(a != b for a, b in zip(tasks, tasks[1:]))


class Evaluator:
    def __init__(self, data: CaseData):
        self.data = data
        self.cache: dict[tuple[int, ...], dict | None] = {}

    def route(self, tasks: list[int]) -> dict | None:
        key = tuple(tasks)
        if key not in self.cache:
            result = schedule_route(tasks, self.data)
            if result is None:
                safe_release = max((zone.end_min for zone in self.data.zones if zone.active), default=0.0)
                result = schedule_route(tasks, self.data, release_min=safe_release)
            self.cache[key] = result
        return self.cache[key]

    def solution(self, routes: list[list[int]]) -> tuple[tuple[float, float], list[dict]] | None:
        schedules = [self.route(route) for route in routes]
        if any(item is None for item in schedules):
            return None
        valid = [item for item in schedules if item is not None]
        times = [item["time_h"] for item in valid]
        key = (max(times), max(times) - min(times), sum(times))
        return key, valid


def lex_better(a: tuple[float, ...], b: tuple[float, ...]) -> bool:
    for x, y in zip(a, b):
        if x < y - TOL:
            return True
        if x > y + TOL:
            return False
    return False


def propose(routes: list[list[int]], times: list[float], rng: random.Random) -> list[list[int]] | None:
    candidate = [route[:] for route in routes]
    move = rng.random()
    order = sorted(range(len(routes)), key=times.__getitem__)

    if move < 0.48:
        src = rng.choice(order[-min(2, len(order)):]) if rng.random() < 0.82 else rng.randrange(len(routes))
        choices = [idx for idx in range(len(routes)) if idx != src]
        dst = rng.choice(order[: min(2, len(order))]) if rng.random() < 0.82 else rng.choice(choices)
        if dst == src:
            dst = rng.choice(choices)
        if len(candidate[src]) <= 1:
            return None
        pos = rng.randrange(len(candidate[src]))
        node = candidate[src].pop(pos)
        legal = [
            q for q in range(len(candidate[dst]) + 1)
            if (q == 0 or candidate[dst][q - 1] != node)
            and (q == len(candidate[dst]) or candidate[dst][q] != node)
        ]
        if not legal:
            return None
        candidate[dst].insert(rng.choice(legal), node)
    elif move < 0.73:
        a, b = rng.sample(range(len(routes)), 2)
        ia, ib = rng.randrange(len(candidate[a])), rng.randrange(len(candidate[b]))
        candidate[a][ia], candidate[b][ib] = candidate[b][ib], candidate[a][ia]
    else:
        ridx = rng.randrange(len(routes))
        route = candidate[ridx]
        if len(route) < 4:
            return None
        i, j = sorted(rng.sample(range(len(route)), 2))
        if i == j:
            return None
        route[i : j + 1] = reversed(route[i : j + 1])

    if not all(valid_tasks(route) for route in candidate):
        return None
    return candidate


def search_seed(
    initial: list[list[int]],
    evaluator: Evaluator,
    seed: int,
    iterations: int,
    deadline: float,
) -> tuple[list[list[int]], tuple[float, float, float], dict]:
    rng = random.Random(seed)
    current = [route[:] for route in initial]
    evaluated = evaluator.solution(current)
    if evaluated is None:
        raise RuntimeError(" 问题二初始路线在问题三调度规则下不可行")
    current_key, current_schedules = evaluated
    best = [route[:] for route in current]
    best_key = current_key
    attempted = accepted = 0

    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        times = [item["time_h"] for item in current_schedules]
        candidate = propose(current, times, rng)
        if candidate is None:
            continue
        attempted += 1
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        accept = lex_better(key, current_key)
        if not accept:
            progress = iteration / max(1, iterations - 1)
            temperature = 0.12 * (1.0 - progress) + 0.002
            current_score = current_key[0] + 0.18 * current_key[1] + 0.002 * current_key[2]
            candidate_score = key[0] + 0.18 * key[1] + 0.002 * key[2]
            loss = max(0.0, candidate_score - current_score)
            accept = rng.random() < math.exp(-loss / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key, schedules
            accepted += 1
            if lex_better(key, best_key):
                best, best_key = [route[:] for route in candidate], key

    return best, best_key, {
        "seed": seed,
```

```
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "Tmax_h": best_key[0],
        "delta_h": best_key[1],
        "total_work_h": best_key[2],
    }


def validate_counts(routes: list[list[int]], data: CaseData) -> None:
    found = Counter(node for route in routes for node in route)
    assert dict(found) == data.expected_visits
    assert all(valid_tasks(route) for route in routes)


def zone_summary(data: CaseData) -> dict:
    active = [zone for zone in data.zones if zone.active]
    involved = {
        node
        for node, point in data.coordinates.items()
        if node != 0 and any(point_inside(point, zone) for zone in active)
    }
    return {
        "zone_count": len(data.zones),
        "positive_duration_count": len(active),
        "zero_duration_count": len(data.zones) - len(active),
        "involved_point_count": len(involved),
        "involved_points": sorted(involved),
    }


def solve_case(
    case_name: str,
    data: CaseData,
    initial: list[list[int]],
    seeds: int,
    iterations: int,
    seconds: float,
) -> dict:
    validate_counts(initial, data)
    evaluator = Evaluator(data)
    baseline = evaluator.solution(initial)
    if baseline is None:
        for idx, route in enumerate(initial, start=1):
            if evaluator.route(route) is None:
                print(f"{case_name} UAV{idx} diagnostic", flush=True)
                schedule_route(route, data, debug=True)
        raise RuntimeError(f"{case_name}: 初始路线无法生成合法动态调度")
    best_routes = [route[:] for route in initial]
    best_key, _ = baseline
    history = []
    deadline = time.monotonic() + seconds
    for offset in range(seeds):
        routes, key, report = search_seed(
            initial,
            evaluator,
            20260818 + offset,
            iterations,
            deadline,
        )
        history.append(report)
        if lex_better(key, best_key):
            best_routes, best_key = routes, key
        if time.monotonic() >= deadline:
            break

    validate_counts(best_routes, data)
    final = evaluator.solution(best_routes)
    assert final is not None
    key, schedules = final
    routes_output = []
    for uav, schedule in enumerate(schedules, start=1):
        routes_output.append({"uav": uav, **schedule})
    return {
        "N": len(best_routes),
        "Tmax": key[0],
        "Tmin": min(item["time_h"] for item in schedules),
        "delta": key[1],
        "total_work_h": key[2],
        "total_wait_min": sum(item["wait_time_h"] * 60 for item in schedules),
        "latest_return": min_to_clock(key[0] * 60),
        "time_origin": "08:00",
        "deadline_17_00": False,
        "objective_order": ["Tmax", "delta", "total_work_h"],
        "zone_summary": zone_summary(data),
        "routes": routes_output,
        "search_history": history,
    }


def write_csv(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary.csv").open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.writer(handle)
        writer.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h", "wait_min", "latest_return"])
        for case_name, case in results.items():
            writer.writerow([case_name, case["N"], case["Tmax"], case["Tmin"], case["delta"], case["total_wait_min"], case["latest_return"]])
    for case_name, case in results.items():
        with (output_dir / f"{case_name}_routes.csv").open("w", newline="", encoding="utf-8-sig") as handle:
            writer = csv.writer(handle)
            writer.writerow(["UAV", "Sequence", "Distance_km", "Service_Count", "Wait_min", "Work_h", "Return"])
            for route in case["routes"]:
                writer.writerow([route["uav"], "-".join(map(str, route["sequence"])), route["distance_km"], route["service_count"], route["wait_time_h"] * 60, route["time_h"], route["return_clock"]])


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--q2-solution", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--seeds", type=int, default=5)
    parser.add_argument("--iterations", type=int, default=25000)
    parser.add_argument("--seconds-per-case", type=float, default=35.0)
    args = parser.parse_args()

    cases = load_cases(args.points, args.zones)
    q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
    assert set(cases) == set(q2)
    results = {}
    for case_name, data in cases.items():
        initial = [[int(x) for x in route["sequence"][1:-1]] for route in q2[case_name]["routes"]]
        started = time.monotonic()
        results[case_name] = solve_case(
            case_name,
            data,
            initial,
            args.seeds,
            args.iterations,
            args.seconds_per_case,
        )
        elapsed = time.monotonic() - started
        case = results[case_name]
        print(
            f"{case_name}: N={case['N']}, Tmax={case['Tmax']:.6f} h, "
            f"Tmin={case['Tmin']:.6f} h, delta={case['delta']:.6f} h, "
            f"wait={case['total_wait_min']:.3f} min, elapsed={elapsed:.1f}s",
            flush=True,
        )

    args.output_dir.mkdir(parents=True, exist_ok=True)
    output = args.output_dir / "q3_solution.json"
    output.write_text(json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8")
    write_csv(results, args.output_dir)
    print(output)
```

```

if __name__ == "__main__":
    main()
```

问题三算法 B 主程序

路径：submission_package/question3/program/q3_solver_enhanced.py

```
""" 问题三算法 B：时变圆形禁飞区下的分层增强求解器。

与基准算法的区别：
1. 每次巡检使用唯一任务副本编号；
2. 对受阻转场比较“等待后直飞”和“边界离散可见图绕飞”；
3. 阶段 A 只搜索最小 Tmax，阶段 B 在给定 epsilon 容差内搜索最小 delta；
4. 固定问题二机队规模是正式方案，额外无人机仅作为敏感性分析。

圆边界用外扩采样环及直线弦近似。采样环半径除以 cos(pi/m)，保证相邻弦
不进入原禁飞圆内部；所有最终飞行事件仍由解析线段—圆—时间区间复核。
"""

from __future__ import annotations

import argparse
import csv
import heapq
import json
import math
import random
import time
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path

import q3_solver as baseline


TOL = 1.0e-9


@dataclass(frozen=True)
class GeoNode:
    label: str
    xy: tuple[float, float]


def task_point(task_id: str) -> int:
    return int(task_id.split("#", 1)[0][1:])


def build_unique_copies(initial_physical: list[list[int]]) ->
    tuple[list[list[str]], dict[str, int]]:
    counters: defaultdict[int, int] = defaultdict(int)
    mapping: dict[str, int] = {}
    routes: list[list[str]] = []
    for route in initial_physical:
        copies: list[str] = []
        for point in route:
            counters[point] += 1
            task_id = f"P{point:03d}#{counters[point]}"
            mapping[task_id] = point
            copies.append(task_id)
        routes.append(copies)
    return routes, mapping


def expected_copy_ids(data: baseline.CaseData) -> set[str]:
    return {
        f"P{point:03d}#{copy_no}"
        for point, count in data.expected_visits.items()
        for copy_no in range(1, count + 1)
    }


def valid_copy_route(route: list[str], mapping: dict[str, int]) ->
    bool:
    if not route:
        return False
    physical = [mapping[item] for item in route]
    return all(a != b for a, b in zip(physical, physical[1:]))


def validate_copy_routes(
    routes: list[list[str]], data: baseline.CaseData, mapping:
    dict[str, int]
) -> None:
    flat = [item for route in routes for item in route]
    assert len(flat) == len(set(flat)), " 任务副本编号重复"
    assert set(flat) == expected_copy_ids(data), " 任务副本缺失或多余"
    assert set(mapping) == expected_copy_ids(data)
    assert all(valid_copy_route(route, mapping) for route in routes)


def point_segment_distance(
    p: tuple[float, float], a: tuple[float, float], b: tuple[float,
    float]
) -> float:
    dx, dy = b[0] - a[0], b[1] - a[1]
    denom = dx * dx + dy * dy
    if denom <= TOL:
        return math.dist(p, a)
    lam = ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / denom
    lam = min(1.0, max(0.0, lam))
    q = (a[0] + lam * dx, a[1] + lam * dy)
    return math.dist(p, q)


class DetourRouter:
    def __init__(
        self,
        data: baseline.CaseData,
        ring_samples: int = 16,
        safety_margin: float = 0.0,
    ) -> None:
        if ring_samples < 8:
            raise ValueError("ring_samples 至少为 8")
        self.data = data
        self.ring_samples = ring_samples
        self.safety_margin = safety_margin
        self.path_cache: dict[tuple[int, int, tuple[int, ...]],
        list[GeoNode] | None] = {}

    def _zone_radius(self, zone: baseline.Zone) -> float:
        return zone.radius + self.safety_margin

    def _static_clear(
        self,
        a: tuple[float, float],
        b: tuple[float, float],
        obstacle_indices: tuple[int, ...],
    ) -> bool:
        for index in obstacle_indices:
            zone = self.data.zones[index]
            radius = self._zone_radius(zone)
            if point_segment_distance(zone.center, a, b) < radius -
            1.0e-8:
                return False
        return True

    def _ring_nodes(self, obstacle_indices: tuple[int, ...]) ->
    list[GeoNode]:
        nodes: list[GeoNode] = []
        inflation = 1.0 / math.cos(math.pi / self.ring_samples)
        for index in obstacle_indices:
            zone = self.data.zones[index]
            radius = self._zone_radius(zone) * inflation + 1.0e-6
            for sample in range(self.ring_samples):
                angle = 2.0 * math.pi * sample / self.ring_samples
                xy = (
                    zone.center[0] + radius * math.cos(angle),
                    zone.center[1] + radius * math.sin(angle),
                )
                # 位于其他同时考虑的圆内部的采样点不能作为安全转向点。
                if any(
                    other != index
                    and math.dist(xy, self.data.zones[other].center)
                    < self._zone_radius(self.data.zones[other]) -
                    1.0e-8
```

```
                    for other in obstacle_indices
                ):
                    continue
                nodes.append(GeoNode(f"Z{index + 1}:B{sample}", xy))
        return nodes

    def _static_detour(
        self,
        a_id: int,
        b_id: int,
        obstacles: tuple[int, ...],
    ) -> list[GeoNode] | None:
        key = (a_id, b_id, obstacles)
        if key in self.path_cache:
            return self.path_cache[key]
        source = GeoNode(f"P{a_id}", self.data.coordinates[a_id])
        target = GeoNode(f"P{b_id}", self.data.coordinates[b_id])
        nodes = [source, target, *self._ring_nodes(obstacles)]
        if any(
            math.dist(source.xy, self.data.zones[index].center)
            < self._zone_radius(self.data.zones[index]) - 1.0e-8
            or math.dist(target.xy, self.data.zones[index].center)
            < self._zone_radius(self.data.zones[index]) - 1.0e-8
            for index in obstacles
        ):
            self.path_cache[key] = None
            return None

        adjacency: list[list[tuple[int, float]]] = [[] for _ in
        ↪  nodes]
        for i in range(len(nodes)):
            for j in range(i + 1, len(nodes)):
                if self._static_clear(nodes[i].xy, nodes[j].xy,
                ↪  obstacles):
                    distance = math.dist(nodes[i].xy, nodes[j].xy)
                    adjacency[i].append((j, distance))
                    adjacency[j].append((i, distance))

        dist = [math.inf] * len(nodes)
        previous = [-1] * len(nodes)
        dist[0] = 0.0
        heap: list[tuple[float, int]] = [(0.0, 0)]
        while heap:
            current, u = heapq.heappop(heap)
            if current > dist[u] + TOL:
                continue
            if u == 1:
                break
            for v, weight in adjacency[u]:
                candidate = current + weight
                if candidate < dist[v] - TOL:
                    dist[v] = candidate
                    previous[v] = u
                    heapq.heappush(heap, (candidate, v))
        if not math.isfinite(dist[1]):
            self.path_cache[key] = None
            return None
        order: list[int] = []
        cursor = 1
        while cursor >= 0:
            order.append(cursor)
            if cursor == 0:
                break
            cursor = previous[cursor]
        path = [nodes[index] for index in reversed(order)]
        self.path_cache[key] = path
        return path

    def _simulate_polyline(
        self,
        nodes: list[GeoNode],
        ready: float,
        service_at_target: bool,
        source_is_base: bool,
        mode: str,
    ) -> dict | None:
        now = ready
        total_distance = 0.0
        events: list[dict] = []
        for index, (left, right) in enumerate(zip(nodes, nodes[1:])):
            distance_km = math.dist(left.xy, right.xy) *
            ↪  baseline.UNIT_KM
            travel_min = distance_km / baseline.SPEED_KMH * 60.0
            last = index == len(nodes) - 2
            blocks = baseline.departure_blocks(
                left.xy,
                right.xy,
                travel_min,
                service_at_target and last,
                self.data.zones,
            )
            depart, reasons = baseline.earliest_departure(now,
            ↪  blocks)
            allow_base_wait = source_is_base and index == 0
            node_number = 0 if allow_base_wait else -1
            if not baseline.wait_is_safe(node_number, left.xy, now,
            ↪  depart, self.data.zones):
                return None
            if depart > now + TOL:
                events.append(
                    {
                        "type": "wait",
                        "node_label": left.label,
                        "node_xy": list(left.xy),
                        "start_min": now,
                        "end_min": depart,
                        "duration_min": depart - now,
                        "reason_zone_ids": reasons,
                        "mode": mode,
                    }
                )
            arrival = depart + travel_min
            events.append(
                {
                    "type": "flight",
                    "from_label": left.label,
                    "to_label": right.label,
                    "from_xy": list(left.xy),
                    "to_xy": list(right.xy),
                    "start_min": depart,
                    "end_min": arrival,
                    "duration_min": travel_min,
                    "distance_km": distance_km,
                    "mode": mode,
                }
            )
            total_distance += distance_km
            now = arrival
        return {
            "arrival_min": now,
            "distance_km": total_distance,
            "events": events,
            "mode": mode,
        }

    def travel(
        self,
        a_id: int,
        b_id: int,
        ready: float,
        service_at_target: bool,
    ) -> dict | None:
        direct_nodes = [
            GeoNode(f"P{a_id}", self.data.coordinates[a_id]),
            GeoNode(f"P{b_id}", self.data.coordinates[b_id]),
        ]
        direct = self._simulate_polyline(
            direct_nodes,
            ready,
            service_at_target,
            a_id == 0,
            "direct_or_wait",
        )
        if direct is not None and not any(event["type"] == "wait" for
        ↪  event in direct["events"]):
            return direct

        horizon = direct["arrival_min"] if direct is not None else
        ↪  max(
            (zone.end_min for zone in self.data.zones if
            ↪  zone.active), default=ready
        )
        obstacles = tuple(
```

```
            index
            for index, zone in enumerate(self.data.zones)
            if zone.active
            and baseline.overlaps(ready, max(ready + TOL, horizon), zone.start_min, zone.end_min)
        )
        detour = None
        if obstacles:
            path = self._static_detour(a_id, b_id, obstacles)
            if path is not None and len(path) > 2:
                detour = self._simulate_polyline(
                    path,
                    ready,
                    service_at_target,
                    a_id == 0,
                    "boundary_visibility_detour",
                )
        candidates = [item for item in (direct, detour) if item is not None]
        if not candidates:
            return None
        return min(candidates, key=lambda item: (item["arrival_min"], item["distance_km"]))


def _schedule_copy_route_with_detours(
    route: list[str],
    mapping: dict[str, int],
    data: baseline.CaseData,
    router: DetourRouter,
) -> dict | None:
    if not valid_copy_route(route, mapping):
        return None
    now = 0.0
    distance_km = 0.0
    events: list[dict] = []
    physical = [mapping[item] for item in route]
    point_sequence = [0, *physical, 0]
    task_sequence = ["BASE", *route, "BASE"]
    for leg, (a_id, b_id) in enumerate(zip(point_sequence, point_sequence[1:]), start=1):
        movement = router.travel(a_id, b_id, now, b_id != 0)
        if movement is None:
            return None
        for event in movement["events"]:
            event = {"leg": leg, **event}
            events.append(event)
        now = movement["arrival_min"]
        distance_km += movement["distance_km"]
        if b_id != 0:
            task_id = route[leg - 1]
            events.append(
                {
                    "type": "service",
                    "task_id": task_id,
                    "point_id": b_id,
                    "node_xy": list(data.coordinates[b_id]),
                    "start_min": now,
                    "end_min": now + baseline.SERVICE_MIN,
                    "duration_min": baseline.SERVICE_MIN,
                }
            )
            now += baseline.SERVICE_MIN
    wait_min = sum(event["duration_min"] for event in events if event["type"] == "wait")
    detour_legs = len(
        {
            event["leg"]
            for event in events
            if event["type"] == "flight" and event.get("mode") == "boundary_visibility_detour"
        }
    )
    return {
        "sequence": point_sequence,
        "task_sequence": task_sequence,
        "distance_km": distance_km,
        "service_count": len(route),
        "flight_time_h": distance_km / baseline.SPEED_KMH,
        "wait_time_h": wait_min / 60.0,
        "time_h": now / 60.0,
        "return_clock": baseline.min_to_clock(now),
        "detour_leg_count": detour_legs,
        "used_baseline_handoff_fallback": False,
        "events": events,
    }


def _baseline_handoff_fallback(
    route: list[str], mapping: dict[str, int], data: baseline.CaseData
) -> dict | None:
    """ 当逐航段绕飞评价陷入不安全驻留时，使用基准算法的反向等待传播。"""
    physical = [mapping[item] for item in route]
    scheduled = baseline.schedule_route(physical, data)
    if scheduled is None:
        safe_release = max((zone.end_min for zone in data.zones if zone.active), default=0.0)
        scheduled = baseline.schedule_route(physical, data, release_min=safe_release)
    if scheduled is None:
        return None
    service_cursor = 0
    converted: list[dict] = []
    for event in scheduled["events"]:
        item = dict(event)
        if event["type"] == "flight":
            a_id, b_id = int(event["from"]), int(event["to"])
            item.update(
                {
                    "from_label": f"P{a_id}",
                    "to_label": f"P{b_id}",
                    "from_xy": list(data.coordinates[a_id]),
                    "to_xy": list(data.coordinates[b_id]),
                    "mode": "direct_with_backward_handoff",
                }
            )
        elif event["type"] == "wait":
            node = int(event["node"])
            item.update(
                {
                    "node_label": f"P{node}",
                    "node_xy": list(data.coordinates[node]),
                    "mode": "direct_with_backward_handoff",
                }
            )
        elif event["type"] == "service":
            item.update(
                {
                    "task_id": route[service_cursor],
                    "point_id": int(event["node"]),
                    "node_xy": list(data.coordinates[int(event["node"])]),
                }
            )
            service_cursor += 1
        converted.append(item)
    return {
        **{key: value for key, value in scheduled.items() if key != "events"},
        "task_sequence": ["BASE", *route, "BASE"],
        "detour_leg_count": 0,
        "used_baseline_handoff_fallback": True,
        "events": converted,
    }


def schedule_copy_route(
    route: list[str],
    mapping: dict[str, int],
    data: baseline.CaseData,
    router: DetourRouter,
) -> dict | None:
    enhanced = _schedule_copy_route_with_detours(route, mapping, data, router)
    if enhanced is not None:
        return enhanced
    return _baseline_handoff_fallback(route, mapping, data)


class EnhancedEvaluator:
    def __init__(
        self,
```

```
        data: baseline.CaseData,
        mapping: dict[str, int],
        ring_samples: int,
        safety_margin: float,
    ) -> None:
        self.data = data
        self.mapping = mapping
        self.router = DetourRouter(data, ring_samples, safety_margin)
        self.cache: dict[tuple[str, ...], dict | None] = {}

    def route(self, tasks: list[str]) -> dict | None:
        key = tuple(tasks)
        if key not in self.cache:
            self.cache[key] = schedule_copy_route(tasks,
            ↪  self.mapping, self.data, self.router)
        return self.cache[key]

    def solution(self, routes: list[list[str]]) -> tuple[tuple[float,
    ↪  float, float], list[dict]] | None:
        schedules = [self.route(route) for route in routes]
        if any(item is None for item in schedules):
            return None
        valid = [item for item in schedules if item is not None]
        times = [item["time_h"] for item in valid]
        return (max(times), max(times) - min(times), sum(times)),
        ↪  valid


def propose(
    routes: list[list[str]],
    times: list[float],
    mapping: dict[str, int],
    rng: random.Random,
) -> list[list[str]] | None:
    candidate = [route[:] for route in routes]
    order = sorted(range(len(routes)), key=times.__getitem__)
    move = rng.random()
    if move < 0.42:
        src = rng.choice(order[-min(2, len(order)):]) if rng.random()
        ↪  < 0.8 else rng.randrange(len(routes))
        choices = [idx for idx in range(len(routes)) if idx != src]
        dst = rng.choice(order[: min(2, len(order))]) if rng.random()
        ↪  < 0.8 else rng.choice(choices)
        if len(candidate[src]) <= 1:
            return None
        task = candidate[src].pop(rng.randrange(len(candidate[src])))
        point = mapping[task]
        legal = [
            pos
            for pos in range(len(candidate[dst]) + 1)
            if (pos == 0 or mapping[candidate[dst][pos - 1]] !=
            ↪  point)
            and (pos == len(candidate[dst]) or
            ↪  mapping[candidate[dst][pos]] != point)
        ]
        if not legal:
            return None
        candidate[dst].insert(rng.choice(legal), task)
    elif move < 0.68:
        a, b = rng.sample(range(len(routes)), 2)
        ia, ib = rng.randrange(len(candidate[a])),
        ↪  rng.randrange(len(candidate[b]))
        candidate[a][ia], candidate[b][ib] = candidate[b][ib],
        ↪  candidate[a][ia]
    elif move < 0.84:
        ridx = rng.randrange(len(routes))
        if len(candidate[ridx]) < 4:
            return None
        i, j = sorted(rng.sample(range(len(candidate[ridx])), 2))
        candidate[ridx][i : j + 1] = reversed(candidate[ridx][i : j +
        ↪  1])
    else:
        a, b = rng.sample(range(len(routes)), 2)
        if len(candidate[a]) < 2 or len(candidate[b]) < 2:
            return None
        ia, ja = sorted(rng.sample(range(len(candidate[a]) + 1), 2))
        ib, jb = sorted(rng.sample(range(len(candidate[b]) + 1), 2))
        if ia == ja or ib == jb:
            return None
        seg_a, seg_b = candidate[a][ia:ja], candidate[b][ib:jb]
        candidate[a][ia:ja], candidate[b][ib:jb] = seg_b, seg_a
    if not all(valid_copy_route(route, mapping) for route in
    ↪  candidate):
        return None
    return candidate


def stage_a_search(
    initial: list[list[str]],
    evaluator: EnhancedEvaluator,
    mapping: dict[str, int],
    rng: random.Random,
    iterations: int,
    deadline: float,
) -> tuple[list[list[str]], tuple[float, float, float], dict]:
    current = [route[:] for route in initial]
    evaluated = evaluator.solution(current)
    if evaluated is None:
        raise RuntimeError("增强算法无法调度初始路线")
    current_key, current_schedules = evaluated
    best, best_key = [route[:] for route in current], current_key
    attempted = accepted = 0
    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, [item["time_h"] for item in
        ↪  current_schedules], mapping, rng)
        if candidate is None:
            continue
        attempted += 1
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        progress = iteration / max(1, iterations - 1)
        temperature = 0.10 * (1.0 - progress) + 0.001
        loss = key[0] - current_key[0]
        accept = loss < -TOL or rng.random() < math.exp(-max(0.0,
        ↪  loss) / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key,
            ↪  schedules
            accepted += 1
            if key[0] < best_key[0] - TOL or (
                abs(key[0] - best_key[0]) <= TOL and key[2] <
                ↪  best_key[2] - TOL
            ):
                best, best_key = [route[:] for route in candidate],
                ↪  key
    return best, best_key, {
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "Tmax_star_h": best_key[0],
        "secondary_metrics_not_optimized": {"delta_h": best_key[1],
        ↪  "total_work_h": best_key[2]},
    }


def stage_b_search(
    stage_a_routes: list[list[str]],
    tmax_star: float,
    epsilon: float,
    evaluator: EnhancedEvaluator,
    mapping: dict[str, int],
    rng: random.Random,
    iterations: int,
    deadline: float,
) -> tuple[list[list[str]], tuple[float, float, float], dict]:
    threshold = (1.0 + epsilon) * tmax_star
    current = [route[:] for route in stage_a_routes]
    evaluated = evaluator.solution(current)
    assert evaluated is not None
    current_key, current_schedules = evaluated
    best, best_key = [route[:] for route in current], current_key
    attempted = accepted = rejected_by_threshold = 0
    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, [item["time_h"] for item in
        ↪  current_schedules], mapping, rng)
        if candidate is None:
            continue
        attempted += 1
```

```
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        if key[0] > threshold + TOL:
            rejected_by_threshold += 1
            continue
        current_obj = (current_key[1], current_key[2],
        ↪  current_key[0])
        candidate_obj = (key[1], key[2], key[0])
        progress = iteration / max(1, iterations - 1)
        temperature = 0.06 * (1.0 - progress) + 0.0005
        loss = (candidate_obj[0] - current_obj[0]) + 0.002 * (
            candidate_obj[1] - current_obj[1]
        )
        accept = candidate_obj < current_obj or rng.random() <
        ↪  math.exp(-max(0.0, loss) / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key,
            ↪  schedules
            accepted += 1
            if (key[1], key[2], key[0]) < (best_key[1], best_key[2],
            ↪  best_key[0]):
                best, best_key = [route[:] for route in candidate],
                ↪  key
    return best, best_key, {
        "epsilon": epsilon,
        "Tmax_limit_h": threshold,
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "rejected_by_Tmax_limit": rejected_by_threshold,
        "delta_h": best_key[1],
        "Tmax_h": best_key[0],
        "total_work_h": best_key[2],
    }


def add_uavs(initial: list[list[str]], extra: int) ->
↪  list[list[str]]:
    routes = [route[:] for route in initial]
    for _ in range(extra):
        src = max(range(len(routes)), key=lambda idx:
        ↪  len(routes[idx]))
        if len(routes[src]) <= 1:
            raise ValueError(" 无法继续拆分非空路线")
        midpoint = len(routes[src]) // 2
        new_route = routes[src][midpoint:]
        routes[src] = routes[src][:midpoint]
        routes.append(new_route)
    return routes


def solve_case_n(
    case_name: str,
    data: baseline.CaseData,
    initial: list[list[str]],
    mapping: dict[str, int],
    epsilon: float,
    ring_samples: int,
    safety_margin: float,
    stage_a_iterations: int,
    stage_b_iterations: int,
    seconds: float,
    seed: int,
) -> dict:
    validate_copy_routes(initial, data, mapping)
    evaluator = EnhancedEvaluator(data, mapping, ring_samples,
    ↪  safety_margin)
    started = time.monotonic()
    phase_deadline = started + seconds / 2.0
    rng = random.Random(seed)
    stage_a_routes, stage_a_key, stage_a_report = stage_a_search(
        initial, evaluator, mapping, rng, stage_a_iterations,
        ↪  phase_deadline
    )
    original_stage_a_tmax = stage_a_key[0]
    stage_b_routes, stage_b_key, stage_b_report = stage_b_search(
        stage_a_routes,
        stage_a_key[0],
        epsilon,
        evaluator,
        mapping,
        rng,
        stage_b_iterations,
        started + seconds,
    )
    refinements: list[dict] = []
    # 启发式阶段 B 可能在探索均衡解时发现更小的 Tmax。此时必须将其回代为
    # 新的阶段A当前最好值，并按收紧后的阈值重新执行阶段B，避免目标错位。
    for refinement in range(3):
        if stage_b_key[0] >= stage_a_key[0] - TOL:
            break
        refinements.append(
            {
                "round": refinement + 1,
                "old_Tmax_star_h": stage_a_key[0],
                "new_Tmax_star_h": stage_b_key[0],
            }
        )
        stage_a_routes = [route[:] for route in stage_b_routes]
        stage_a_key = stage_b_key
        stage_b_routes, stage_b_key, stage_b_report = stage_b_search(
            stage_a_routes,
            stage_a_key[0],
            epsilon,
            evaluator,
            mapping,
            rng,
            stage_b_iterations,
            time.monotonic() + max(2.0, seconds / 3.0),
        )
    stage_a_report["initial_Tmax_star_h"] = original_stage_a_tmax
    stage_a_report["Tmax_star_h"] = stage_a_key[0]
    stage_a_report["refined_from_stage_b"] = bool(refinements)
    stage_a_report["refinement_history"] = refinements
    validate_copy_routes(stage_b_routes, data, mapping)
    evaluated = evaluator.solution(stage_b_routes)
    assert evaluated is not None
    key, schedules = evaluated
    assert key[0] <= (1.0 + epsilon) * stage_a_key[0] + TOL
    return {
        "case": case_name,
        "N": len(stage_b_routes),
        "Tmax": key[0],
        "Tmin": min(item["time_h"] for item in schedules),
        "delta": key[1],
        "total_work_h": key[2],
        "total_wait_min": sum(item["wait_time_h"] * 60.0 for item in
        ↪  schedules),
        "latest_return": baseline.min_to_clock(key[0] * 60.0),
        "time_origin": "08:00",
        "deadline_17_00": False,
        "zone_policy": "half_open; zero-duration zones excluded",
        "task_copy_policy": "unique task IDs Pxxx#copy_no",
        "detour_model": {
            "type": "sampled boundary visibility graph",
            "ring_samples": ring_samples,
            "safety_margin_coordinate_units": safety_margin,
        },
        "stage_a": stage_a_report,
        "stage_b": stage_b_report,
        "routes": [
            {"uav": uav, **schedule}
            for uav, schedule in enumerate(schedules, start=1)
        ],
    }


def write_summary(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary_enhanced.csv").open("w", newline="",
    ↪  encoding="utf-8-sig") as handle:
        writer = csv.writer(handle)
        writer.writerow(
            [
                "Case",
                "Scenario",
                "N",
                "StageA_TmaxStar_h",
                "Epsilon",
                "Final_Tmax_h",
                "Final_Tmin_h",
                "Final_delta_h",
```

```
                "Wait_min",
                "Detour_legs",
                "Latest_return",
            ]
        )
        for case_name, scenarios in results.items():
            for scenario_name, item in scenarios.items():
                writer.writerow(
                    [
                        case_name,
                        scenario_name,
                        item["N"],
                        item["stage_a"]["Tmax_star_h"],
                        item["stage_b"]["epsilon"],
                        item["Tmax"],
                        item["Tmin"],
                        item["delta"],
                        item["total_wait_min"],
                        sum(route["detour_leg_count"] for route in
                        ↪  item["routes"]),
                        item["latest_return"],
                    ]
                )


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--q2-solution", type=Path, required=True)
    parser.add_argument(
        "--baseline-q3-solution",
        type=Path,
        help="可选：算法A的 q3_solution.json;
        ↪  提供后作为算法B的必选初始种子",
    )
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--epsilon", type=float, default=0.005)
    parser.add_argument("--ring-samples", type=int, default=16)
    parser.add_argument("--safety-margin", type=float, default=0.0)
    parser.add_argument("--stage-a-iterations", type=int,
    ↪  default=800)
    parser.add_argument("--stage-b-iterations", type=int,
    ↪  default=800)
    parser.add_argument("--seconds-per-case", type=float,
    ↪  default=45.0)
    parser.add_argument("--extra-uavs", type=int, default=0)
    args = parser.parse_args()
    if not 0.0 <= args.epsilon <= 0.05:
        raise ValueError("epsilon 建议位于 [0,0.05]")
    if args.extra_uavs < 0:
        raise ValueError("extra-uavs 不能为负")

    cases = baseline.load_cases(args.points, args.zones)
    q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
    assert set(cases) == set(q2)
    baseline_q3 = None
    if args.baseline_q3_solution is not None:
        baseline_q3 = json.loads(args.baseline_q3_solution.read_text
        ↪  (encoding="utf-8"))
        assert set(cases) == set(baseline_q3)
    results: dict[str, dict] = {}
    for case_index, (case_name, data) in enumerate(cases.items()):
        seed_source = baseline_q3[case_name] if baseline_q3 is not
        ↪  None else q2[case_name]
        physical = [
            [int(value) for value in route["sequence"][1:-1]]
            for route in seed_source["routes"]
        ]
        initial, mapping = build_unique_copies(physical)
        expected = expected_copy_ids(data)
        if set(mapping) != expected:
            raise RuntimeError(f"{case_name}:
            ↪  问题二路线与任务副本需求不一致")
        scenarios: dict[str, dict] = {}
        scenario_routes = [route[:] for route in initial]
        for extra in range(args.extra_uavs + 1):
            scenario_name = "formal_fixed_N0" if extra == 0 else
            ↪  f"sensitivity_N0_plus_{extra}"
            result = solve_case_n(
                case_name,
                data,
                scenario_routes,
                mapping,
                args.epsilon,
                args.ring_samples,
                args.safety_margin,
                args.stage_a_iterations,
                args.stage_b_iterations,
                args.seconds_per_case,
                20260818 + 1000 * case_index + extra,
            )
            scenarios[scenario_name] = result
            print(
                f"{case_name} {scenario_name}: N={result['N']} "
                f"Tmax*={result['stage_a']['Tmax_star_h']:.6f} "
                f"Tmax={result['Tmax']:.6f}
                ↪  delta={result['delta']:.6f} "
                f"wait={result['total_wait_min']:.3f}",
                flush=True,
            )
            # 机队规模敏感性采用逐级热启动：
            ↪  下一规模从当前已优化方案拆出一条
            # 新路线，而不是每次回到未经优化的原始路线重新硬拆。
            if extra < args.extra_uavs:
                optimized_routes = [route["task_sequence"][1:-1] for
                ↪  route in result["routes"]]
                scenario_routes = add_uavs(optimized_routes, 1)
        results[case_name] = scenarios

    args.output_dir.mkdir(parents=True, exist_ok=True)
    output = args.output_dir / "q3_enhanced_solution.json"
    output.write_text(json.dumps(results, ensure_ascii=False,
    ↪  indent=2), encoding="utf-8")
    write_summary(results, args.output_dir)
    print(output)


if __name__ == "__main__":
    main()
```

问题三快速独立校验程序

路径：submission_package/question3/program/q3_fast_validator.py

```
""" 问题三保存结果的独立快速复核。

先用线段包围盒与圆包围盒作常数时间筛查，仅对可能相交的航段求解析交区间；
再逐事件核验飞行、服务和等待均不与正时长管制区冲突。
"""

from __future__ import annotations

import argparse
import json
import math
from collections import Counter
from datetime import datetime
from pathlib import Path

import pandas as pd


REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-7


def to_min(text: object) -> float:
    value = datetime.strptime(str(text).strip(), "%H:%M")
    return (value.hour - 8) * 60 + value.minute


def overlap(a0: float, a1: float, b0: float, b1: float) -> bool:
    return max(a0, b0) < min(a1, b1) - TOL


```

```python
def bbox_maybe(a, b, center, radius) -> bool:
    return not (
        max(a[0], b[0]) < center[0] - radius
        or min(a[0], b[0]) > center[0] + radius
        or max(a[1], b[1]) < center[1] - radius
        or min(a[1], b[1]) > center[1] + radius
    )


def segment_inside_interval(a, b, center, radius):
    if not bbox_maybe(a, b, center, radius):
        return None
    dx, dy = b[0] - a[0], b[1] - a[1]
    fx, fy = a[0] - center[0], a[1] - center[1]
    aa = dx * dx + dy * dy
    bb = 2 * (fx * dx + fy * dy)
    cc = fx * fx + fy * fy - radius * radius
    disc = bb * bb - 4 * aa * cc
    if disc < -TOL:
        return None
    disc = max(0.0, disc)
    lo = max(0.0, (-bb - math.sqrt(disc)) / (2 * aa))
    hi = min(1.0, (-bb + math.sqrt(disc)) / (2 * aa))
    return None if lo > hi + TOL else (lo, hi)


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--solution", type=Path, required=True)
    args = parser.parse_args()

    result = json.loads(args.solution.read_text(encoding="utf-8"))
    points_book = pd.ExcelFile(args.points)
    zones_book = pd.ExcelFile(args.zones)
    assert set(result) == set(points_book.sheet_names) ==
    ↪  set(zones_book.sheet_names)

    for case_name in points_book.sheet_names:
        points_df = pd.read_excel(points_book, sheet_name=case_name)
        zones_df = pd.read_excel(zones_book, sheet_name=case_name)
        coords = {0: (0.0, 0.0)}
        expected = {}
        for _, row in points_df.iterrows():
            point_id = int(row.Point_ID)
            coords[point_id] = (float(row.X_Coordinate),
            ↪  float(row.Y_Coordinate))
            expected[point_id] = REQ[str(row.Inspection_Level)]
        zones = []
        for _, row in zones_df.iterrows():
            start, end = to_min(row.Start_Time), to_min(row.End_Time)
            if end > start + TOL:
                zones.append((str(row.Zone_ID), (float(row.Center_X),
                ↪  float(row.Center_Y)), float(row.Radius), start,
                ↪  end))

        found = Counter()
        times = []
        for route in result[case_name]["routes"]:
            seq = [int(x) for x in route["sequence"]]
            assert seq[0] == seq[-1] == 0
            assert all(a != b for a, b in zip(seq[1:-1], seq[2:-1]))
            found.update(seq[1:-1])
            last_end = 0.0
            for event in route["events"]:
                start, end = float(event["start_min"]),
                ↪  float(event["end_min"])
                assert abs(start - last_end) < 2.0e-6
                assert end >= start - TOL
                if event["type"] == "flight":
                    a, b = coords[int(event["from"])],
                    ↪  coords[int(event["to"])]
                    for zone_id, center, radius, z0, z1 in zones:
                        interval = segment_inside_interval(a, b,
                        ↪  center, radius)
                        if interval is None:
                            continue
                        lo, hi = interval
                        assert not overlap(start + lo * (end -
                        ↪  start), start + hi * (end - start), z0,
                        ↪  z1), f"{case_name} UAV{route['uav']}
                        ↪  flight conflicts {zone_id}"
                elif event["type"] in {"service", "wait"}:
                    node = int(event["node"])
                    if event["type"] == "wait" and node == 0:
                        last_end = end
                        continue
                    point = coords[node]
                    for zone_id, center, radius, z0, z1 in zones:
                        if math.dist(point, center) <= radius + TOL:
                            assert not overlap(start, end, z0, z1),
                            ↪  f"{case_name} UAV{route['uav']}
                            ↪  {event['type']} conflicts {zone_id}"
                last_end = end
            assert abs(last_end / 60 - float(route["time_h"])) <
            ↪  2.0e-6
            times.append(last_end / 60)
        assert dict(found) == expected
        case = result[case_name]
        assert int(case["N"]) == len(case["routes"])
        assert abs(max(times) - float(case["Tmax"])) < 2.0e-6
        assert abs(min(times) - float(case["Tmin"])) < 2.0e-6
        assert abs(max(times) - min(times) - float(case["delta"])) <
        ↪  2.0e-6
        print(f"{case_name}: PASSED")


if __name__ == "__main__":
    main()
```

问题三算法 B 逐事件独立验证程序

路径：submission_package/question3/program/q3_enhanced_validator.py

```python
""" 问题三增强算法的独立逐事件验证器。"""

from __future__ import annotations

import argparse
import json
from collections import Counter
from pathlib import Path

import q3_solver as baseline
from q3_solver_enhanced import expected_copy_ids, task_point


TOL = 2.0e-6


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--solution", type=Path, required=True)
    args = parser.parse_args()

    cases = baseline.load_cases(args.points, args.zones)
    result = json.loads(args.solution.read_text(encoding="utf-8"))
    assert set(result) == set(cases)
    for case_name, scenarios in result.items():
        data = cases[case_name]
        expected = expected_copy_ids(data)
        for scenario_name, case in scenarios.items():
            found_tasks: list[str] = []
            times: list[float] = []
            total_wait = 0.0
            for route in case["routes"]:
                task_sequence = route["task_sequence"]
                assert task_sequence[0] == task_sequence[-1] ==
                ↪  "BASE"
                tasks = task_sequence[1:-1]
                found_tasks.extend(tasks)
```

```
            physical = [task_point(task) for task in tasks]
            assert route["sequence"] == [0, *physical, 0]
            assert all(a != b for a, b in zip(physical,
            ↪  physical[1:]))

            last_end = 0.0
            services: list[str] = []
            distance_km = 0.0
            for event in route["events"]:
                start = float(event["start_min"])
                end = float(event["end_min"])
                assert abs(start - last_end) < TOL
                assert end >= start - TOL
                if event["type"] == "flight":
                    a = tuple(float(x) for x in event["from_xy"])
                    b = tuple(float(x) for x in event["to_xy"])
                    distance_km += float(event["distance_km"])
                    for zone in data.zones:
                        if not zone.active:
                            continue
                        interval =
                        ↪  baseline.segment_circle_interval(a,
                        ↪  b, zone)
                        if interval is None:
                            continue
                        lo, hi = interval
                        assert not baseline.overlaps(
                            start + lo * (end - start),
                            start + hi * (end - start),
                            zone.start_min,
                            zone.end_min,
                        ), f"{case_name}/{scenario_name}/UAV{rou
                        ↪  te['uav']} flight conflicts
                        ↪  {zone.zone_id}"
                elif event["type"] in {"wait", "service"}:
                    xy = tuple(float(x) for x in
                    ↪  event["node_xy"])
                    if event["type"] == "wait":
                        total_wait += end - start
                    else:
                        services.append(event["task_id"])
                        assert abs((end - start) -
                        ↪  baseline.SERVICE_MIN) < TOL
                    for zone in data.zones:
                        if zone.active and
                        ↪  baseline.point_inside(xy, zone):
                            assert not baseline.overlaps(start,
                            ↪  end, zone.start_min,
                            ↪  zone.end_min), (
                                f"{case_name}/{scenario_name}/UA
                                ↪  V{route['uav']} "
                                f"{event['type']} conflicts
                                ↪  {zone.zone_id}"
                            )
                else:
                    raise AssertionError(f"未知事件类型
                    ↪  {event['type']}")
                last_end = end
            assert services == tasks
            assert abs(distance_km - float(route["distance_km"]))
            ↪  < TOL
            assert abs(last_end / 60.0 - float(route["time_h"]))
            ↪  < TOL
            times.append(float(route["time_h"]))

        assert len(found_tasks) == len(set(found_tasks))
        assert set(found_tasks) == expected
        assert int(case["N"]) == len(case["routes"])
        assert abs(max(times) - float(case["Tmax"])) < TOL
        assert abs(min(times) - float(case["Tmin"])) < TOL
        assert abs(max(times) - min(times) -
        ↪  float(case["delta"])) < TOL
        assert abs(total_wait - float(case["total_wait_min"])) <
        ↪  TOL
        limit = float(case["stage_b"]["Tmax_limit_h"])
        assert float(case["Tmax"]) <= limit + TOL
        assert case["deadline_17_00"] is False
        print(f"{case_name}/{scenario_name}: PASSED")


if __name__ == "__main__":
    main()
```